

Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

Implementierung und Matlab-Praxis 6:

Simulink

Inhalt

6. Simulink

- 6.1 Grundlagen von Simulink
- 6.2 Simulink-Standard-Bibliothek
- 6.3 Modellstrukturierung
- 6.4 Simulationsverfahren/-parameter
- 6.5 Benutzerdefinierte Blöcke und Bibliotheken
- 6.6 Besonderheiten
- 6.7 Steuern von Simulink aus Matlab
- 6.8 Zusatzbibliotheken

6. Simulink - 6.1 Grundlagen

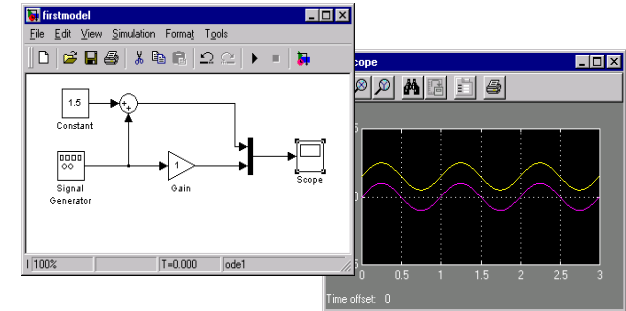
Was ist Simulink?

Antwort:

Simulink ist ein an Matlab gekoppeltes domänenunspezifisches Simulationstool mit grafischer Oberfläche zur Erstellung von Simulationsmodellen als Erweiterung.

Simlink ist speziell für die Simulation bei regelungstechnischen Anwendungen geeignet.

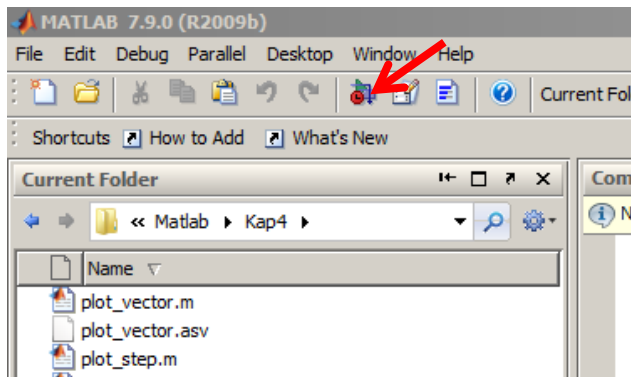
Simulink ist weniger geeignet für spezielle domänenspezifische Simulationen, z. B. Schaltungssimulation



6. Simulink - 6.1 Grundlagen

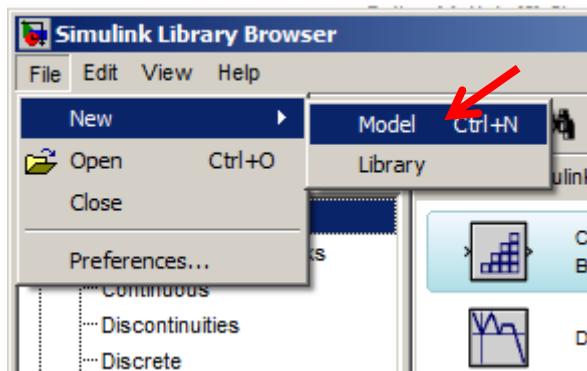
Starten von Simulink:

- Der Befehl `simulink` oder `simulink3` oder Button in Matlab



öffnet die Simulink-Library

- Erzeugen eines neuen Modells: *Menu Bar: File -> New -> Model*

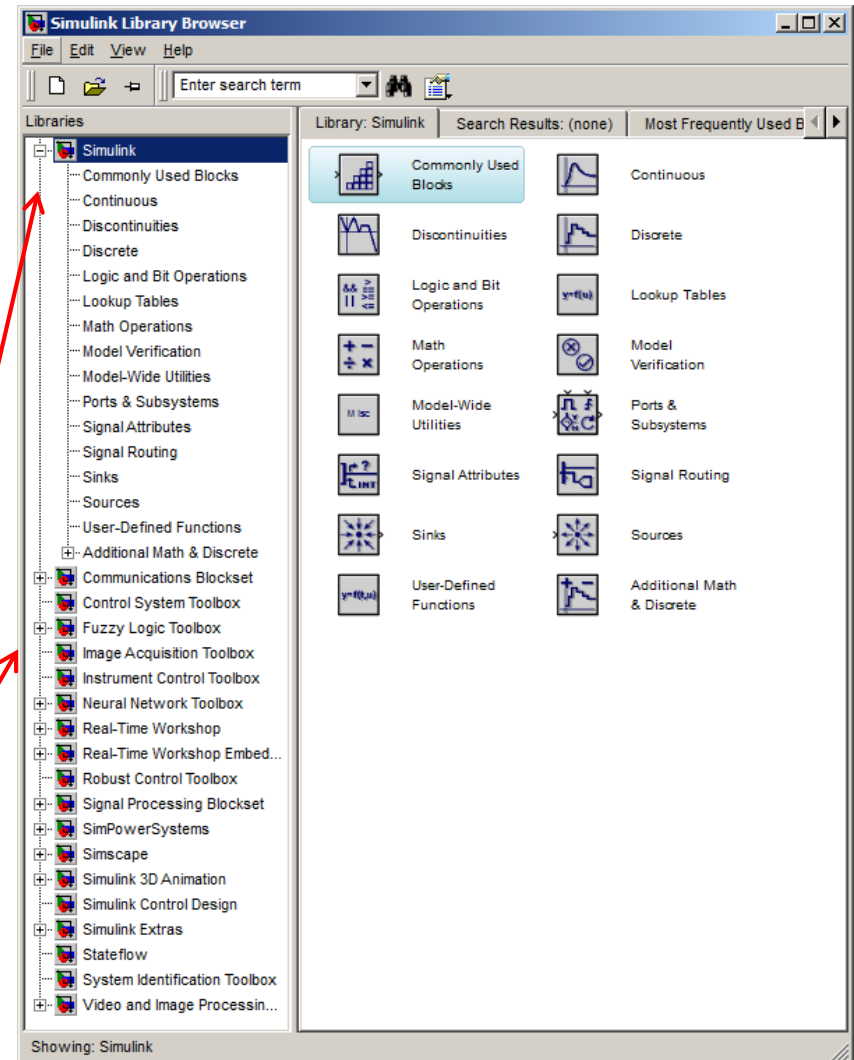


Alternativ:
Erzeugen einer
neuen Library

6. Simulink - 6.1 Grundlagen

Erstellen eines Simulink Modelldiagramms:

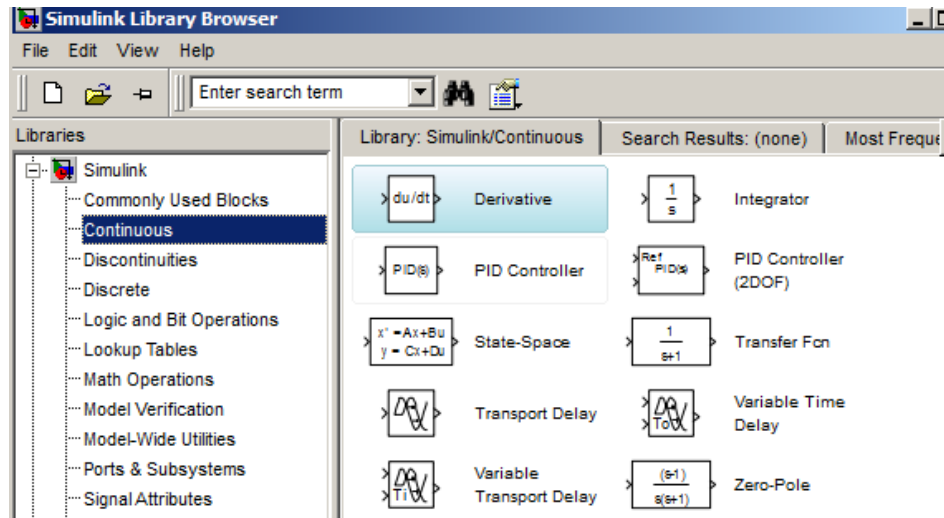
- Blöcke können per „drag and drop“ aus den Bibliotheken ins „Modell“ (Simulink Modelldiagramm) kopiert werden.
- Kopieren im Modell funktioniert über `<ctrl-c>`, `<ctrl-v>` oder mit dem rechten Maus-Button
- Verbindungen können mit dem linken Maus-Button gezogen werden, Abzweige mit dem rechten.
- Simulink-Modelle werden mit der Extension `*.mdl` gespeichert.
- Der Library-Browser zeigt neben der Simulink-Standard-Library alle verfügbaren Zusatz-Library die entweder von TMW (Toolboxen) oder von anderen Anbietern kommen können.



6. Simulink - 6.1 Grundlagen

Simulink-Standard-Libraries:

Continuous: Kontinuierliche lineare dynamische Übertragungsglieder



Discontinuous: Nichtlineare statische Übertragungsglieder
(mit Zero-Crossing detection)

Discrete: Diskrete abtastende dynamische Übertragungsglieder

6. Simulink - 6.1 Grundlagen

Logic and Bit Operations: Logik-Blöcke

Lookup Tables: Kennfeld-Blöcke

Math Operation: Mathematische Funktionsblöcke

Model Verification: Blöcke zur Überprüfung des Modellverhaltens

Model Wide Utilities: Hilfsblöcke

Ports and Subsystems: Subsystemsblöcke zur Gliederung und Steuerung

Signal Attributes: Zuweisung von Signaleigenschaften

Signal Routing: Blöcke für spezielle Verbindungen

6. Simulink - 6.1 Grundlagen

Sinks: Signalsenken

Sources: Signalquellen

User-Defined Systems: Benutzerdefinierte Systeme

6. Simulink - 6.1 Grundlagen

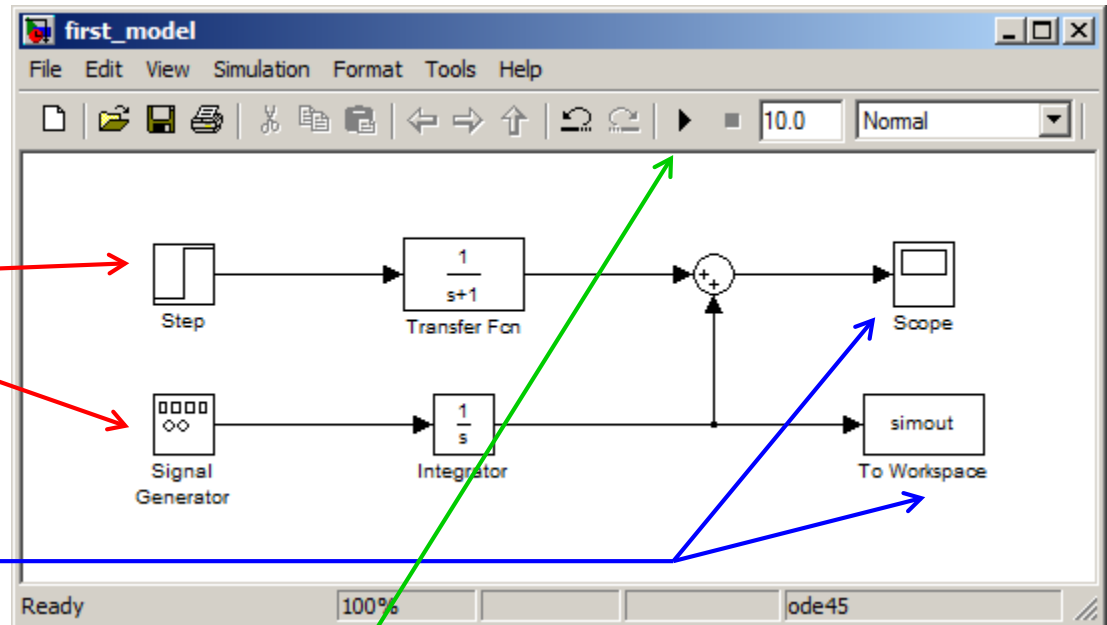
Simulink-Modell:

Ein Simulink-Modell
beinhaltet mindestens einen

Source-Block

und einen

Sink-Block

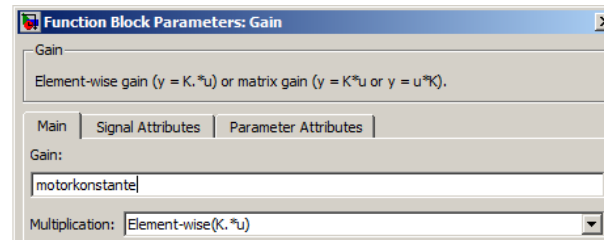
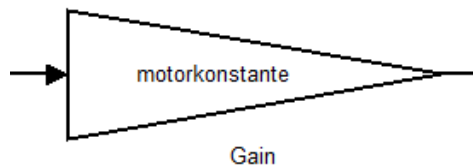


Das **Starten** der Simulation erfolgt über den Start-Button

6. Simulink - 6.1 Grundlagen

Parameter:

- Parameter in Blöcken können als Wert oder als Variable eingetragen werden.



- Bei Variablen-Angaben werden die Werte der aktuellen gleichnamigen Workspace-Variablen bei der Simulation verwendet
- Sind die Variablen auf dem Workspace nicht bekannt, bricht die Simulation mit einer Fehler ab.

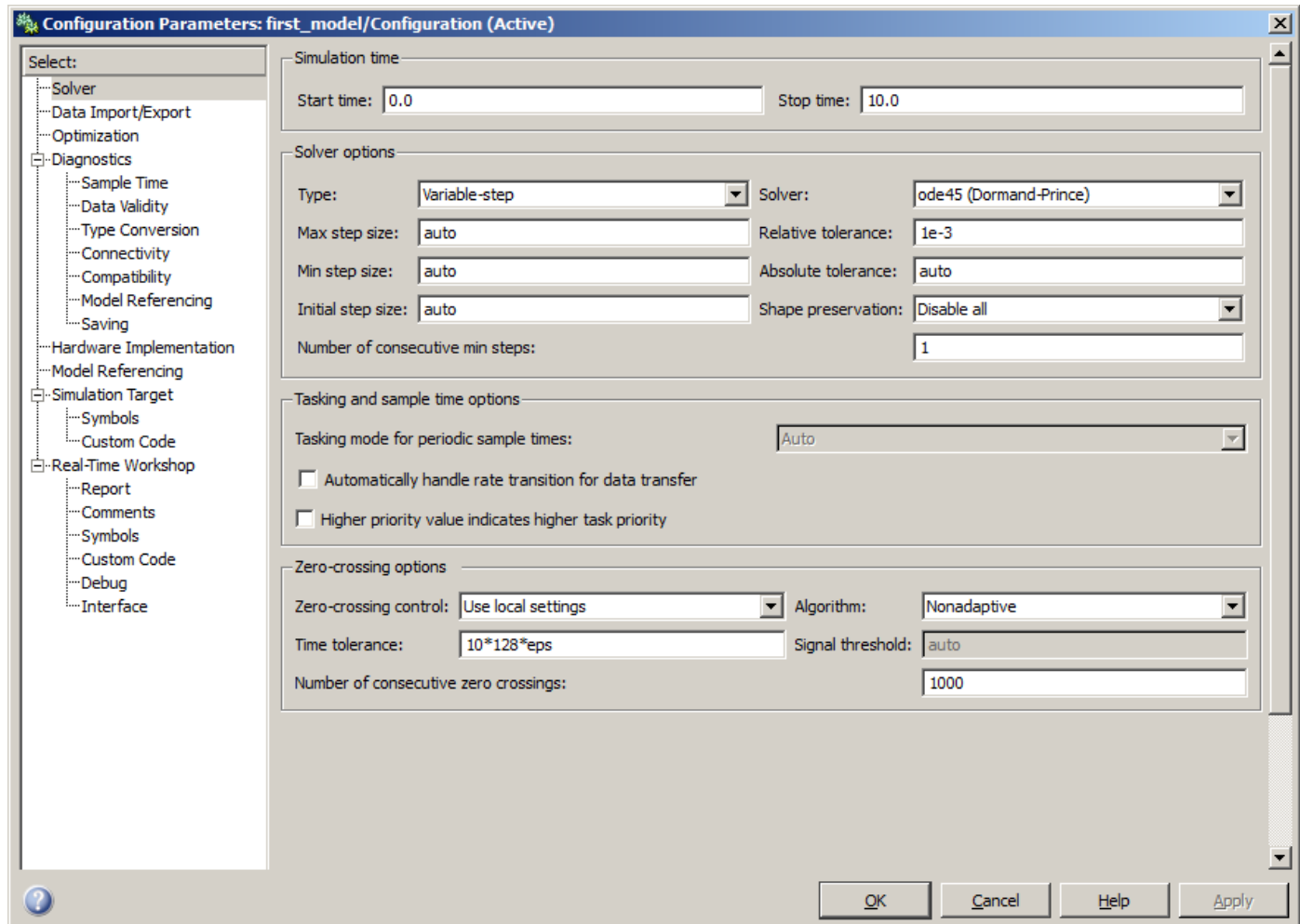
Anmerkung:

Als sinnvolle Arbeitsweise hat sich das „Initialisieren“ der Modell über m-Skripte herausgestellt:

- Dies ermöglicht die Verwendung unterschiedlicher Parametersätze auf ein Modell.
- Einfache Veränderung von Parameter die an mehreren Stellen verwendet werden.
- Parameter lassen sich einfacher finden.
- Feste Größen (z. B. Umrechnungsfaktoren) sollten allerdings direkt ins Modell eingetragen werden.

7. Simulink - Grundlagen

Simulationseinstellungen: (Menü → Simulation → Configure Parameters)



6. Simulink - 6.1 Grundlagen

Wichtige Einstellungen:

- Start- und Stop-Zeit
- Variable oder feste Schrittweite
- Simulationsalgorithmus („Solver“)

6. Simulink - 6.1 Grundlagen

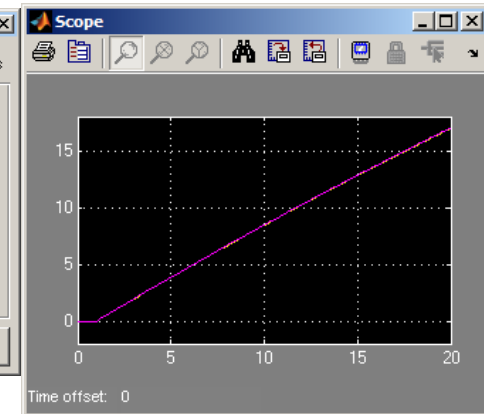
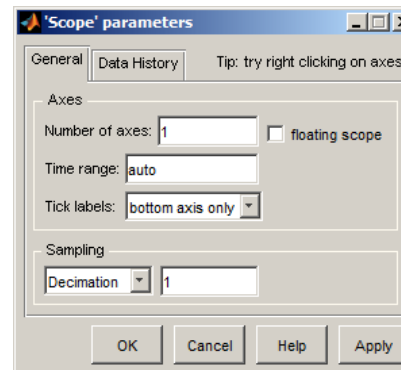
Wichtige Sink-Blöcke:

Scope:

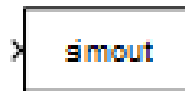


Scope

- Direkte Darstellung von Zeitverläufen aus der Simulation.
- Mehrere Achsen einstellbar.
- Mehrere Größen pro Achse durch MUX-Block möglich.
- Die Anzahl der aufgezeichneten Schritte ist einstellbar (→ Data History).



To Workspace:



To Workspace

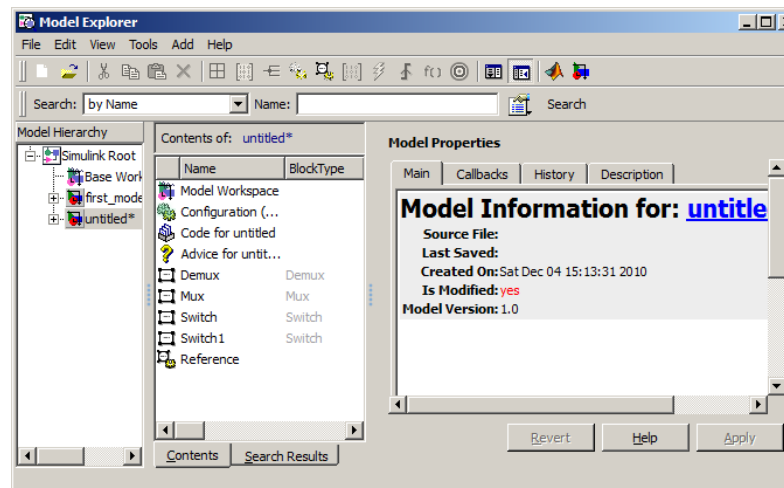
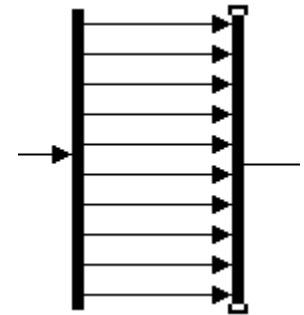
- Ablegen von Simulationdaten als Vektor auf dem Matlab-Workspace.
- Der Zeitvektor wird je nach Simulationseinstellung automatisch abgeleitet (t_{out})

Anmerkung: Für die Darstellung von Simulationsergebnissen in Berichten (o. Ä.) empfiehlt es sich die Simulationsdaten mit den Matlab-Plot-Befehlen darzustellen.

6. Simulink - 6.1 Grundlagen

Hilfreiches beim Arbeiten mit Simulink:

- Zoomen bei Simlink-Diagrammen → Taste „r“
- Auszoomen bei Simlink-Diagrammen → Taste „v“
- „Toggle-Selection“ von Blöcken bei gehaltener „Shift-Taste“
- Vereinfachtes Verbinden zweier Blöcke:
 - Ersten Block markieren (linke Maustaste).
 - Zweiten Block bei gehaltener „Strg-Taste“ mit rechter Maustaste markieren.
 - Funktioniert auch bei n-fachen Verbindungslinien zwischen den Blöcken.
- „Model Explorer“
 - Gibt ein Überblick über die im Verzeichnis enthaltenen Modelle und ermöglicht das Einsehen und Editieren der Modell „Description“ und „History“.



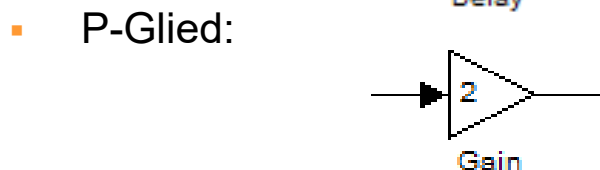
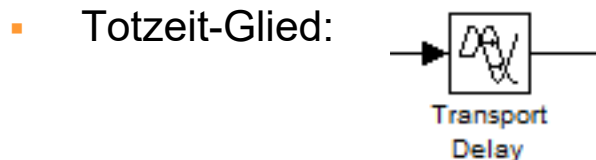
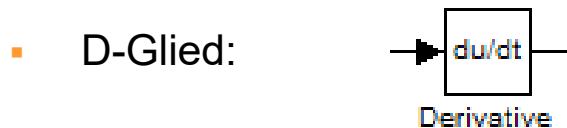
6. Simulink - 6.2 Standard-Block-Bibliothek

Continuous: Kontinuierliche: lineare dynamische Übertragungsglieder:

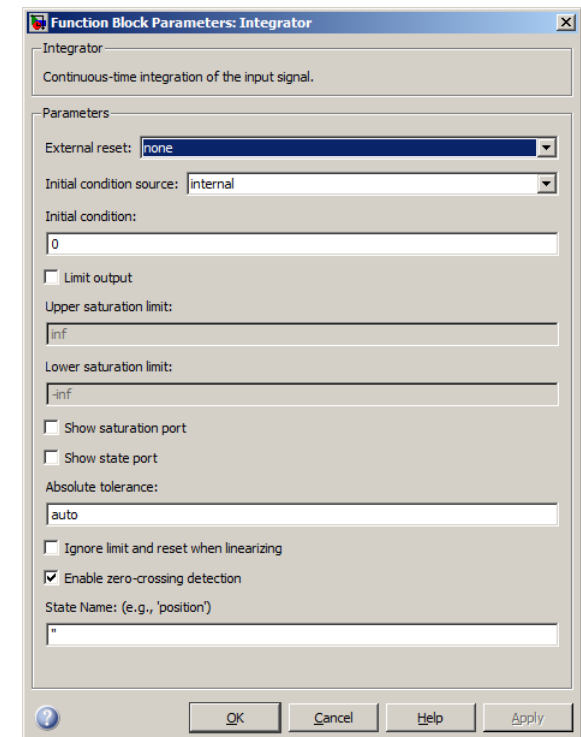
Elementare Übertragungsglieder:



Relative umfangreiche Konfigurationsmöglichkeiten (Sättigung, Zero-Crossing-Detection, Startwert, usw.)



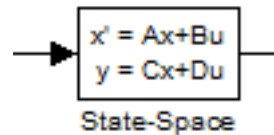
(Ist Teil der „Math Operations“)



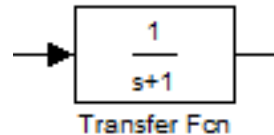
6. Simulink - 6.2 Standard-Block-Bibliothek

Übertragungsfunktionen:

- State-Space-Block:

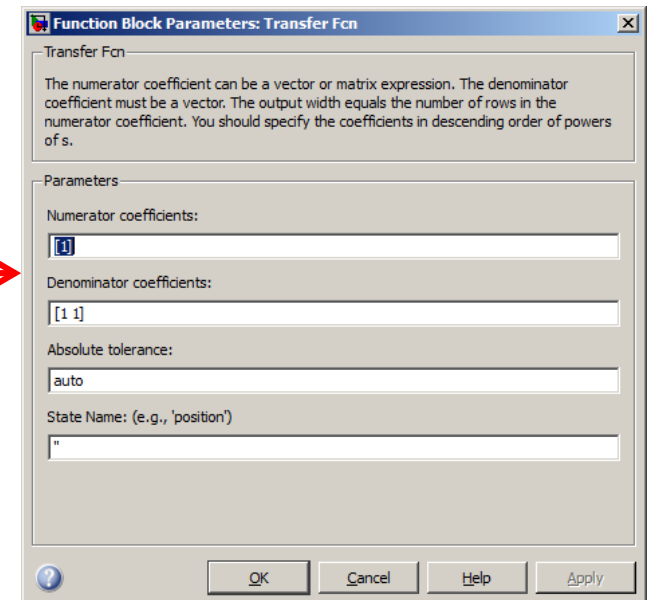
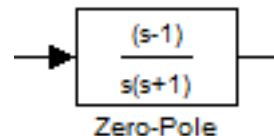


- Übertragungsfunktion



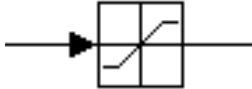
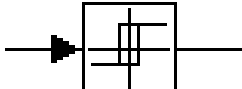
Die Übertragungsfunktion wird durch die Koeffizientenvektoren von Zähler- und Nennerpolynom angegeben

- Übertragungsfunktion in Pol-Nullstellen-Darstellung

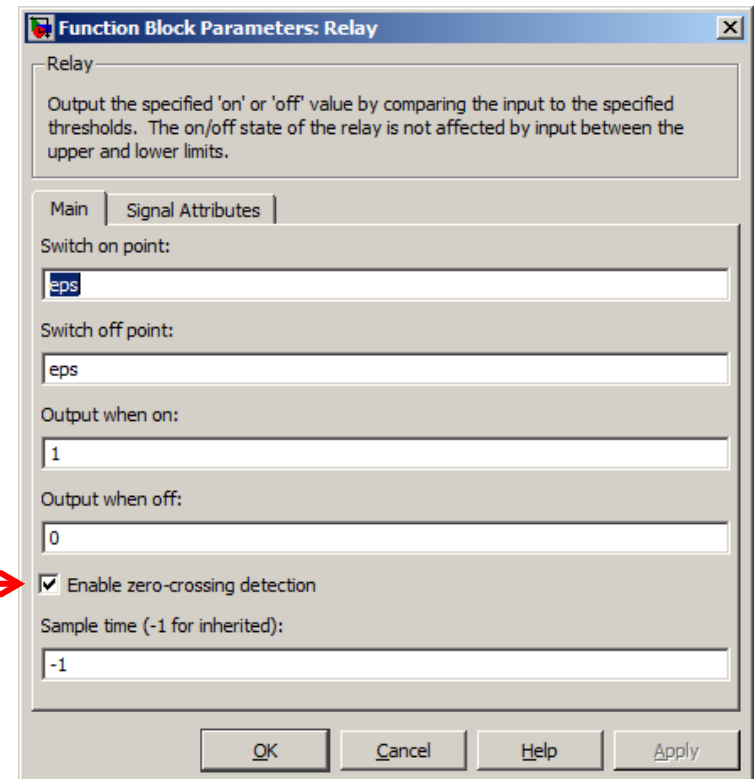


6. Simulink - 6.2 Standard-Block-Bibliothek

Discontinuous: Nichtlineare statische Übertragungsglieder:

- Sättigung:

Saturation
- Schaltfunktion (mit Hysterese)

Relay

Zur wichtigsten Eigenschaft der Blöcke der „Discontinuous“-Lib zählt das „Zero Crossing Detection“ (nur bei variabler Schrittweite): Annäherung an einen Schaltzeitpunkt durch Anpassung der Schrittweite.



6. Simulink - 6.2 Standard-Block-Bibliothek

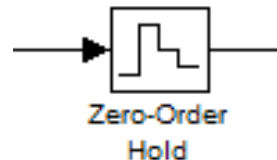
Discrete: Diskrete abtastende dynamische Übertragungsglieder:

- z-Übertragungsfunktion:

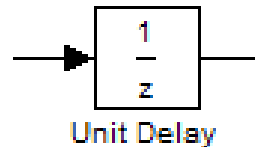


Spezifikation wie bei der Übertragungsfunktion.

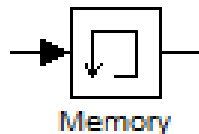
- Abtasthalteglied:
(0-ter Ordnung)



- Diskrete Totzeit



- Memory-Block

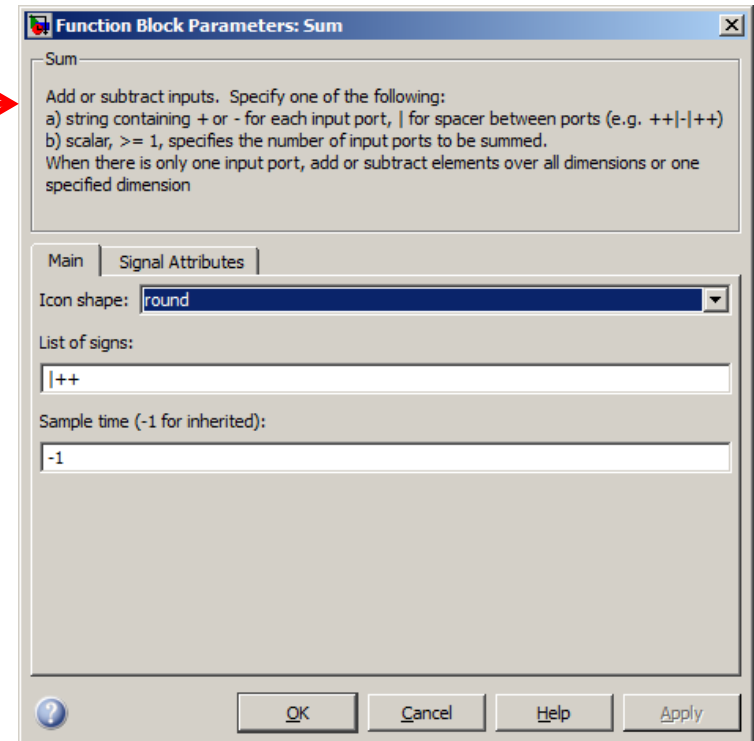
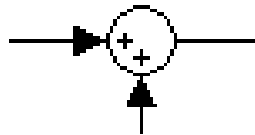


Je nach Einstellungen identische Funktion wie „Unit-Delay“.

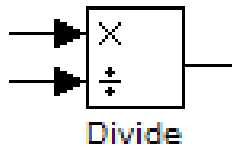
6. Simulink - 6.2 Standard-Block-Bibliothek

Math Operation: Mathematische Funktionsblöcke:

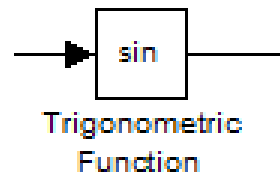
- Addition/Subtraktion:



- Multiplikation/Division:



- Trigonometrische Funktionen:



6. Simulink - 6.2 Standard-Block-Bibliothek

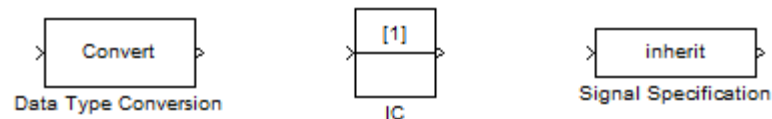
Signale:

Signale können unterschiedliche Eigenschaften aufweisen. Sie unterscheiden sich z. B. in:

- Dimension (es sind auch Vektoren und Matizen möglich)
- Daten-Typ (wie Matlab: double, single, boolean, int32, uint32,...)
- Abtastzeit
- usw...

In der Regel ergeben sich die Signalattribute inherent durch die Blöcke die durch das Signal verbunden sind.

Die „Signal Attributes“ Library enthält Blöcke mit denen die Signalattribute zusätzlich eingestellt werden können:



6. Simulink - 6.3 Modellstrukturierung

Grundsätzliche Anmerkung:

Ähnlich der Verwendung von Funktionen, Prozeduren und Kommentaren muss bei „Grafischer Programmierung“ mit Strukturelementen gearbeitet werden, um eine gewisse „Lesbarkeit“ und „Selbstdokumentation“ der (Modell-)Diagramme zu gewährleisten:

- Simulink-Modelle können einen beachtliche Umfang erreichen.
- Simulink-Modelle sollte aufgrund einer sinnvollen Strukturierung immer weitgehend selbstdokumentierend sein.

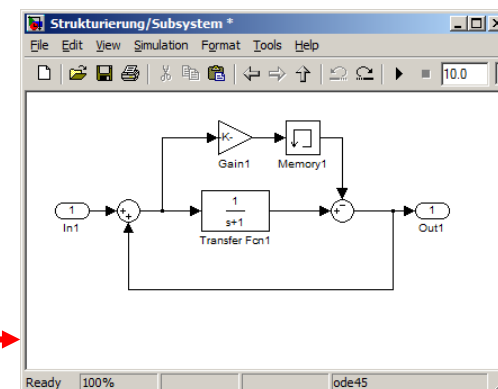
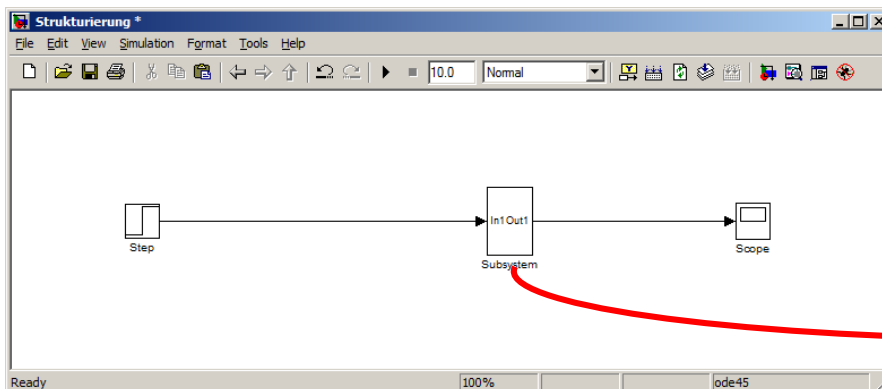
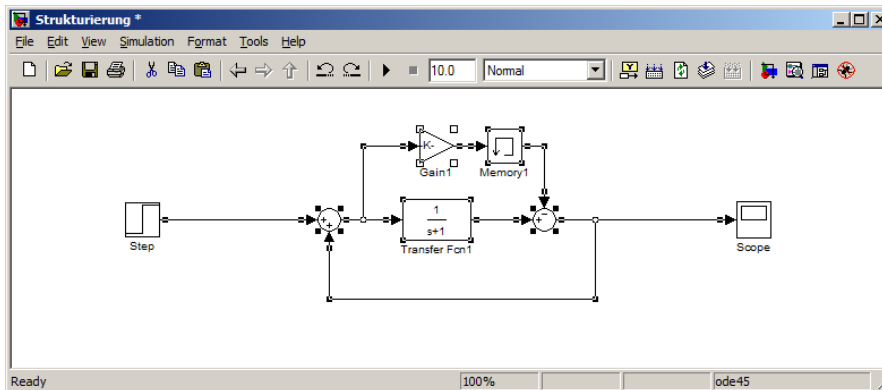
Struktur- und Dokumentationselemente:

- Subsysteme (mit sinnvollen Namen!)
- Spezielle Blöcke für das Signalrouting (Bus-Blöcke, Mux-, Demux-, Goto- u. From-Blk)
- Sinnvolle Blocknamen und Signallabel
- Freier Text (als Kommentar) im Diagramm
- Sinnvolle Formatierungen (Blockfarben)

6. Simulink - 6.3 Modellstrukturierung

Erzeugen eines Subsystems:

- Kopieren aus der Bibliothek
- Markieren eines Teilmodells und rechte Maustaste (→ „Create Subsystem“)

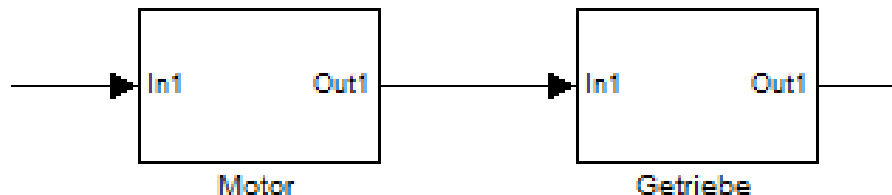


6. Simulink - 6.3 Modellstrukturierung

Arbeiten mit Subsystemen:

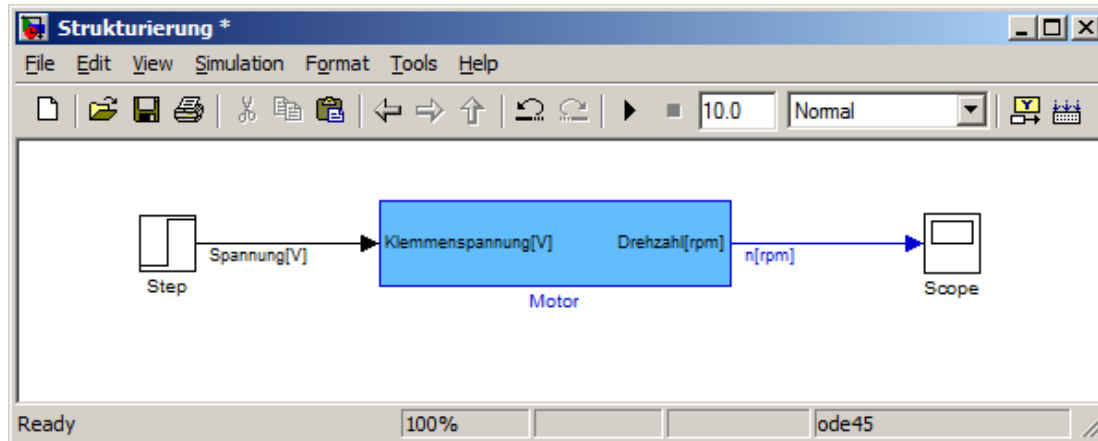
- Subsysteme können sich in (nahezu) beliebiger Tiefe schachteln
- Subsysteme sollten zur Gliederung in logisch sinnvolle und zusammengehörige Teilemodell genutzt werden.
- Bei großen Modellen kann es sinnvoll sein die Parameter des Modells als Struct mit einer der Subsystemgliederung identischen Struktur anzulegen.

```
MDL.Getriebe.Uebersetzung.v = 0.75;  
MDL.Getriebe.Reibung.v = 0.0012;  
MDL.Motor.Leistung.v = 3;
```

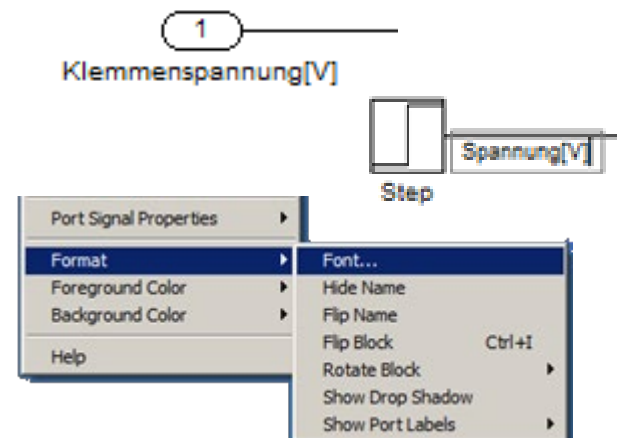


6. Simulink - 6.3 Modellstrukturierung

„Formatieren“ des Simulink-Modelles:



- Umbenennen der Blöcke und Subsysteme → Mausklick auf den Namen
- Portnamen werden durch die Namen der Portblöcke im Subsystem festgelegt
- Benennen (Labeln) von Signalen → Mausklick an die Signallinie.
- Formatierungsoptionen befinden sich Kontextmenü (rechte Maustaste),

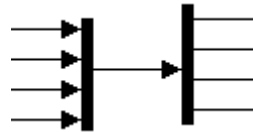


6. Simulink - 6.3 Modellstrukturierung

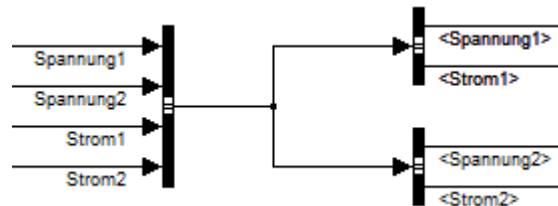
Signalrouting:

Neben dem direkten Verbinden diskreter Signale gibt es:

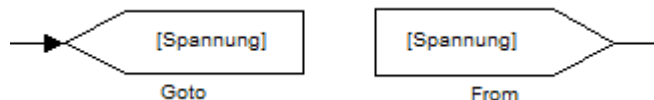
- „Mux“ und Demux“-Blöcke:



- Busse:



- „Goto“- und „From“-Blöcke:



- Alle Blöcke ermöglichen den Mehrfachabgriff.

- Identifikation der Signale über die Ordnung.
- Einzelauswahl möglich
- Hierarchien möglich
- Identifikation der Signale über Signallabel.
- Einzelauswahl möglich
- Hierarchien möglich
- Identifikation der Signale über ein Label in den Blockeinstellungen.
- Kann auch für Busse oder Mux-Signale verwendet werden
- Beim Überspringen von Subsystemgrenzen muss die „Global“ Option gesetzt sein.

6. Simulink - 6.3 Modellstrukturierung

Sinnvolles Verwenden der Signalrouting-Blöcke:

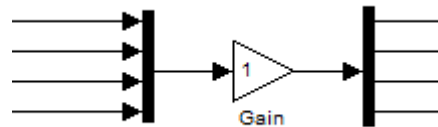
- **Wichtig:** Einfache diskrete Signallinien sind i. d. R. die eingängigste Darstellungsmöglichkeit die ein schnelles Erfassen der Modellstruktur ermöglichen.
- Bus- und Mux-Blöcke verwenden bei
 - Sehr vielen Signalen die zwischen Modellkomponenten verbunden werden müssen.
 - Zum „Sammeln“ von Signalen (z. B. zur Darstellung).
 - Bei gleichartigen oder zusammengehörigen Signalen die einem sinnvollen Bus weitergeführt werden.
- Goto- und From-Blöcke sehr sparsam verwenden: Die Verbindung ohne sichtbare „Leitung“ verhindert zwar ggf. ein unübersichtliche Zahl von Signallinien im Modell, aber auch eine Nachvollziehbarkeit.
→ Nur verwenden wenn die jeweiligen Quell- und Zielsysteme leicht zu identifizieren sind.

6. Simulink - 6.3 Modellstrukturierung

Vektorisierte Funktionen:

Wie auch viele Matlab-Funktionen können viele Simulink-Blöcke auf Vektoren, (und Matrizen) angewendet werden, sind also vektorisiert.

Vektoren entstehen durch Zusammenführen von Signalen gleicher Art per Mux-Block:



Die funktioniert bei einer Vielzahl von Blöcken:

- Mathematischen Operationen
- Übertragung-Gliedern
- Diskontinuitäten
- Bedingtem Routing (Schaltern)
- usw. ...

Wichtig: Die Vektorbreiten müssen kompatibel sein.

6. Simulink - 6.3 Modellstrukturierung

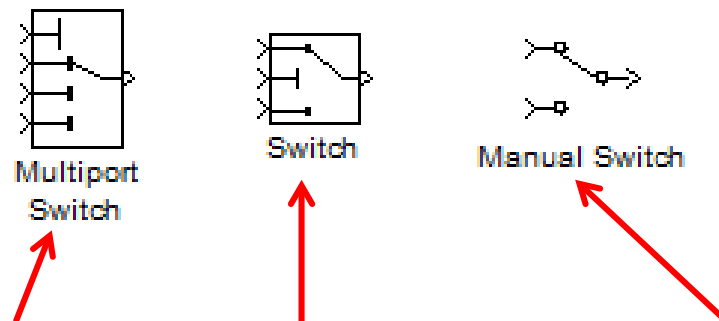
Virtuelle Blöcke:

Normale Subsysteme sowie Mux-, Demux-, Bus-, Goto- und From-Blöcke sind sogenannte „virtuelle“ Blöcke, d. h. reine Strukturelemente, die die Simulation selbst nicht beeinflussen und keine Rechenzeit kosten.

Wichtig:

Nicht alle Signalrouting-Blöcke und Subsysteme sind virtuelle Blöcke:

Offensichtliche Beispiele: Bedingtes Signalrouting (Switches):



Abhängigkeit:

Index

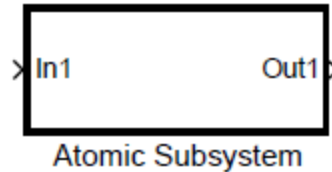
Schwellwert

manuelle Umschaltung

6. Simulink - 6.3 Modellstrukturierung

Besondere Subsysteme:

- *Atomic Subsystem:*



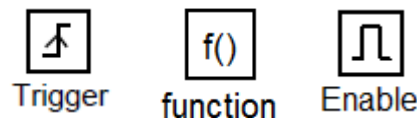
Erzwingt die „geschlossene“ Abarbeitung des beinhaltenden Teilsystem während der Simulation

- *Bedingte Bearbeitung (Signalabhängig)*
- *Bedingte Bearbeitung (Kontrollstrukturen)*

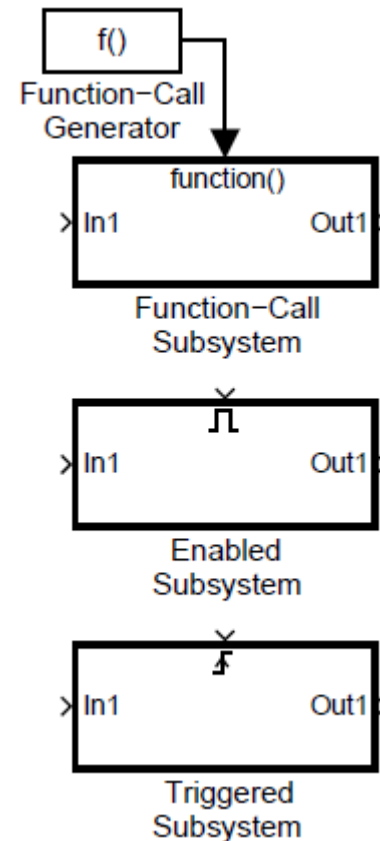
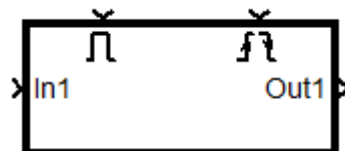
6. Simulink - 6.3 Modellstrukturierung

Bedingte Bearbeitung (Signalabhängig):

- Beinhalten die geschlossene Abarbeitung („Atomic“)
- Typen:
 - Enabled Subsystem
 - Triggered Subsystem
 - Function-called Subsystem
- Erzeugen oder Verändern durch Einfügen des entsprechenden „Trigger“- „Enable“- oder „Function-Call“- Blockes in das Subsystem.



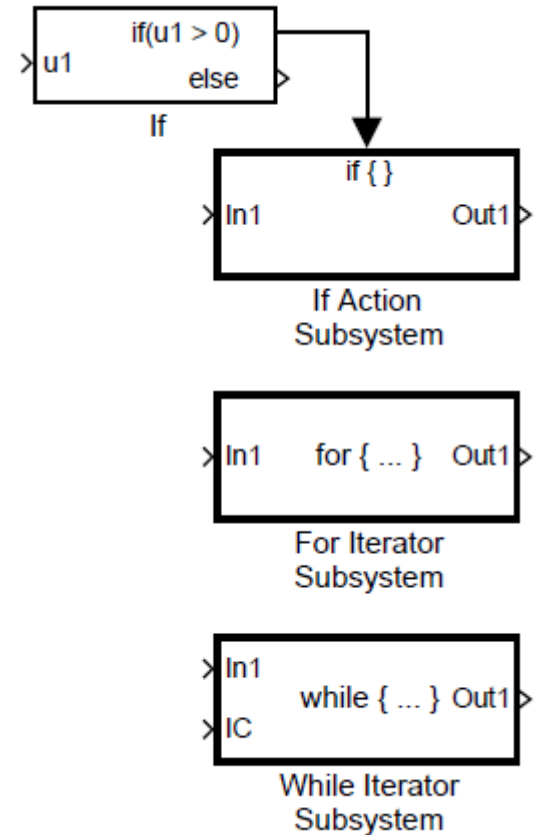
- Kombination möglich



6. Simulink - 6.3 Modellstrukturierung

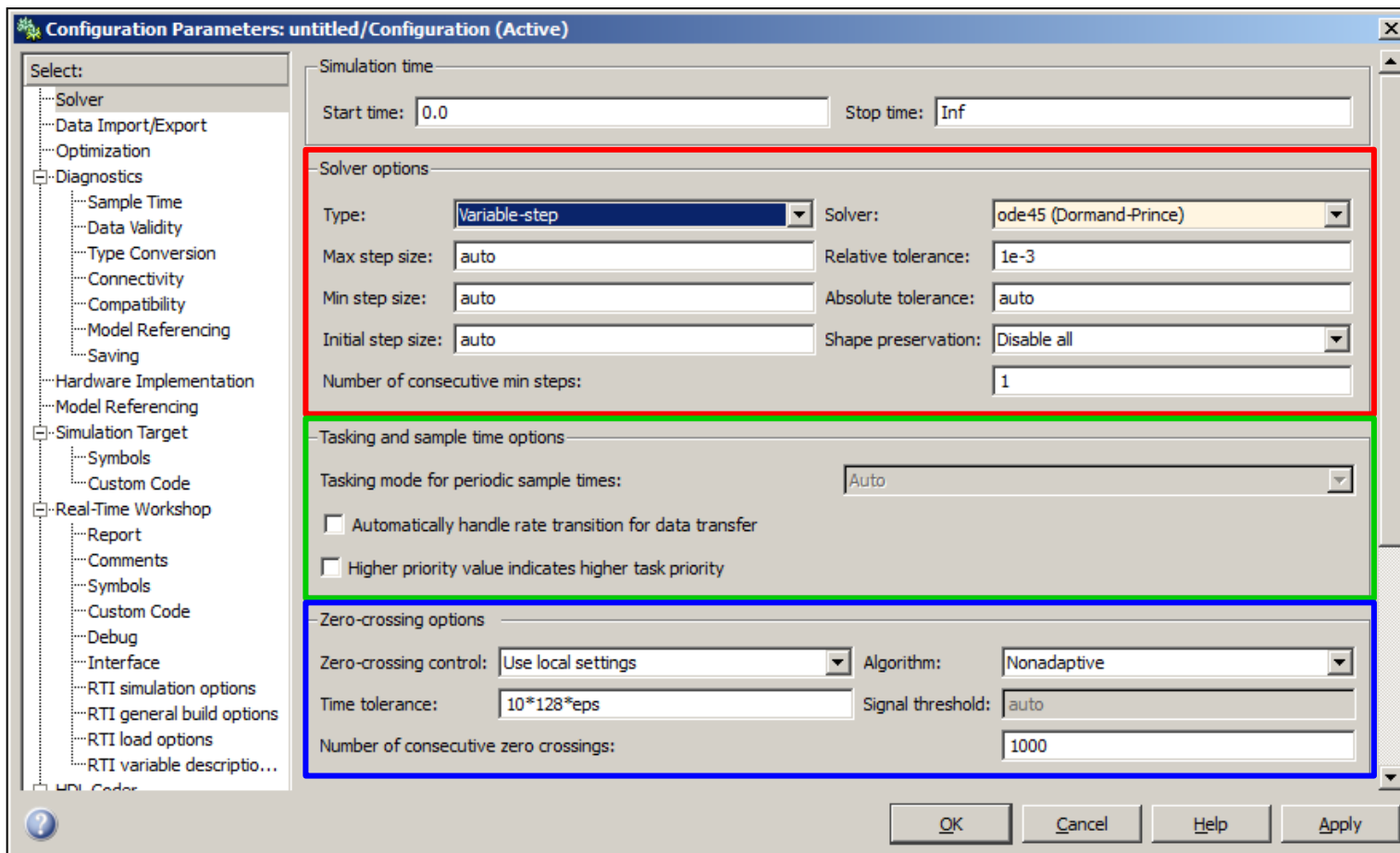
Bedingte Bearbeitung (Kontrollstrukturen):

- Beinhalten die geschlossene Abarbeitung („Atomic“):
- Emuliert Kontrollstrukturen wie sie von Programmiersprachen bekannt sind:
 - If... else -Subsystem
 - Switch... Case -Subsystem
 - For-Iterator-Subsystem
 - While-Iterator-Subsystem
- Erzeugen oder Verändern durch Einfügen des entsprechenden „Trigger“- , „Enable“- oder „Function-Call“- Blockes in das Subsystem.



6. Simulink - 6.4 Simulationsverfahren/-parameter

Matlab/Simulink: Einstellen der Simulationsparameter
→ Simulation → Configuration Parameters → Page: **Solver**



6. Simulink - 6.4 Simulationsverfahren/-parameter

Einstellungen für das Integrationsverfahren:

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Type	Variable-step	Integrationsverfahren mit variabler Schrittweite (default); für den RTW (Codegenerierung für den Echtzeitbetriebe) nicht wählbar.
	Fixed-step	Integrationsverfahren mit fester Schrittweite; Einstellung für den RTW.

Hinweis: Die Seite *Solver* passt sich je nach Einstellung (z. B. Variable-step / Fixed-step oder Wahl des Verfahrens) an. So sind immer nur die Einstellungen zu sehen, die auf das gewählte Verfahren Einfluss haben.

6. Simulink - 6.4 Simulationsverfahren/-parameter

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Solver (Variable-step)	Discrete (no continuous states)	kein Verfahren aktiv
	ode45 (Dormand-Prince)	s.u. (default)
	ode23 (Bogacki-Shampine)	s.u.
	ode113 (Adams)	s.u.
	ode15s (stiff/NDF)	s.u.
	ode23s (stiff/Mod. Rosenbrock)	s.u.
	ode23t (Mod. stiff/Trapezoidal)	s.u.
	ode23tb (stiff/TR-BDF2)	s.u.

6. Simulink - 6.4 Simulationsverfahren/-parameter

Simulink ODE-Suiten (Variable-step):

ode45 (Dormand-Prince): Default-Verfahren

- Runge-Kutta-45
- Explizites Einschrittverfahren

→ Standardverfahren, da erfahrungsgemäß für die meisten Probleme geeignet

ode23 (Bogacki-Shampine):

- Runge-Kutta-23
- Explizites Einschrittverfahren
- Schneller als **ode45**

→ Empfehlenswert zur schnellen Simulation bei unkritischen Problemen

ode113 (Adams)

- Adams-Bashforth-Moulton-Verfahren
- Explizites Mehrschrittverfahren
- Doppelter Korrektorschritt (PECECE)

→ Empfehlenswert bei aufwendigen Systemfunktionen aber nicht-steifen System

6. Simulink - 6.4 Simulationsverfahren/-parameter

ode15s (stiff/NDF)

- Numerical **D**ifferentiation **F**ormulas
Ähnlich BDF bzw. Gear-Verfahren aber explizite
 - Explizites Mehrschrittverfahren mit Ordnungssteuerung
- Empfehlenswert bei steifen Systemen mit wenig Schaltvorgängen.

ode23s (stiff/Mod. Rosenbrock)

- Rosenbrock-Verfahren
 - Linear-implizites Einschrittverfahren
- Empfehlenswert bei sehr steifen Systemen.

ode23t (Mod. stiff/Trapezoidal)

- Basierend auf Trapezregel und Interpolation
 - Implizites Einschrittverfahren
- Empfehlenswert zur schnellen Simulation aber nur bei leicht steifen Systemen.

ode23tb (stiff/TR-BDF2)

- Implizites Runge-Kutta Verfahren (kombiniert mit BDF)
- Empfehlenswert zur schnellen Simulation aber nur bei leicht steifen Systemen.

6. Simulink - 6.4 Simulationsverfahren/-parameter

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Max. Step Size	<Wert in Sek.>	Wert der größten von der Schrittweitensteuerung verwendete Schrittweite
	auto	Wert der wird aus der Simulationsdauer ermittelt.
Min. Step Size	<Wert in Sek.>	Wert der kleinste von der Schrittweitensteuerung verwendet Schrittweite
	[<Wert in Sek.>, <Anzahl Warn.>]	Wert der kleinste verwendet Schrittweit und Anzahl der Warnung für das Erreichen der kleinsten Schrittweite nach denen ein Simulationsabbruch erzwungen wird
	auto	Wert wird aus der Rechengenauigkeit des Systems ermittelt
Initial Step Size	<Wert in Sek.>	Startwert der von der Schrittweitensteuerung verwendet Schrittweite
	auto	Wert wird aus den Ableitung der Systemfunktion bei Simulationsstart ermittelt

6. Simulink - 6.4 Simulationsverfahren/-parameter

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Relative tolerance	<Wert in %>	Wert der relative Toleranz in Prozent von der Zustandsgröße. ¹
Absolute tolerance	<Wert>	Wert der absoluten Toleranz.
	auto	Berechnung aus relativer Toleranz mal max. erreichte Zustandsgröße (Betrag)
Maximum order (nur NDF)	<Wert> (1-5)	Maximale Ordnung für die Ordnungsteuerung des NDF
Solver reset method (² nur ode15s ode23t, ode23tb)	Fast	Keine Neuberechnung der Jacobi-Matrix
	Robust	Neuberechnung der Jacobi-Matrix nach Struktur-Wechseln (Zero Crossing Detection).

¹ Dieser Wert wird für die Schrittweitensteuerung genutzt.

² Verfahren benötigen die Jacobi-Matrix

6. Simulink - 6.4 Simulationsverfahren/-parameter

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Shape preservation	Enable all	Ableitungsinformation wird zur Verbesserung des Simulationsgüte verwendet.
	Disable all	
Tasking mode for periodic sample times	<Wert n>	Anzahl der zulässigen Unterschreitungen der minimalen Schrittweite

6. Simulink - 6.4 Simulationsverfahren/-parameter

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Solver (Fixed-step)	Discrete (no continuous states)	kein Verfahren aktiv
	ode8 (Dormand-Prince)	s.u.
	ode5 (Dormand-Prince)	s.u.
	ode4 (Runge-Kutta)	s.u.
	ode3 (Bogacki-Shampine)	s.u. (default)
	ode2 (Heun)	s.u.
	ode1 (Euler)	s.u.
	ode14x (extrapolation)	s.u.

odeN → Explizite Einschrittverfahren (Namen selbsterklärend)

ode14x (extrapolation) → Implizites Extrapolationsverfahren

6. Simulink - 6.4 Simulationsverfahren/-parameter

Einstellungen für das Taskhandling:

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Tasking mode for periodic sample times (Fixed-step)	Auto	Wahl des Modus in Abhängigkeit von den Einstellungen: Multi Tasking bei unterschiedlichen Abtastzeiten
	SingleTasking	Alle Blöcke werden mit der gleichen Schrittweite gerechnet
	Multi Tasking	Alle Blöcke werden in Abhängigkeit von den Einstellungen mit unterschiedlicher Schrittweite berechnet
Automatically handle rate transition for data transfer	On	Automatisches Einfügen von Rate-Transition-Blöcken zwischen Modellteilen unterschiedlicher Abtastzeit.
	Off	

→ Weiter Einstellung s. Echtzeitsimulation

6. Simulink - 6.4 Simulationsverfahren/-parameter

Einstellungen für das Zero-Crossing-Detection: (Variable-step)

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Zero-crossing control	Use local settings	Zero-Crossing-Detection abhängig von den Blockeinstellungen
	Enable all	Immer Zero-Crossing-Detection
	Disable all	Nie Zero-Crossing-Detection
Time tolerance	<Wert> in Sek.	Zeitliche Toleranz für die Detection von Nulldurchgängen
Number of consecutive zero crossings	<Wert n>	Anzahl der zulässigen in Folge auftretenden Nulldurchgänge
Algorithm	Adaptive	Adaptiven Algorithmus einschalten
	Nonadaptive	

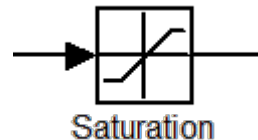
6. Simulink - 6.4 Simulationsverfahren/-parameter

Relevante Blöcke in Simulink (Beispiele):

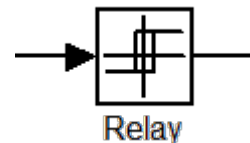
Library: *Discontinuous*

(Nichtlineare statische Übertragungsglieder)

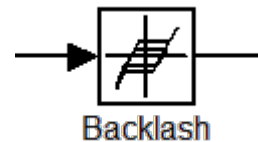
- Sättigung:



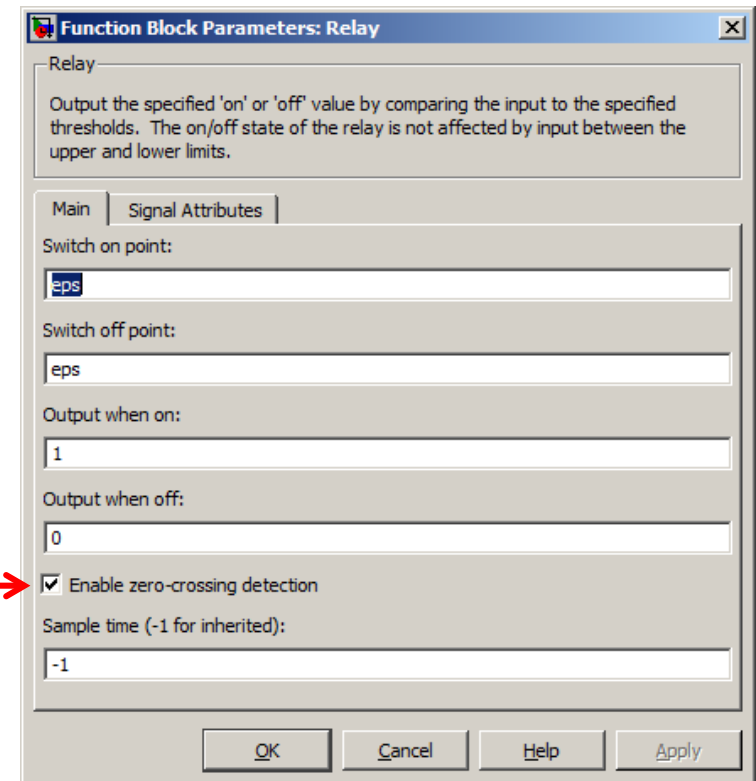
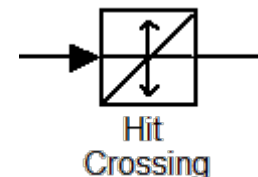
- Schaltfunktion (mit Hysterese)



- Flankenspiel:



- Ereignis Detection:



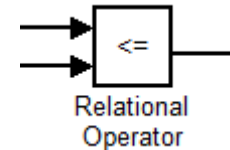
Zur wichtigsten Eigenschaft der Blöcke der „Discontinuous“-Lib zählt das „Zero Crossing Detection“ (nur bei variabler Schrittweite) → Annäherung an einen Schaltzeitpunkt.

→ Weiterführend: Siehe auch Stateflow (von Zustandsdiagrammen).

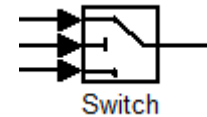
6. Simulink - 6.4 Simulationsverfahren/-parameter

Weiter Blöcke mit ‚Zero Crossing Detektion‘:

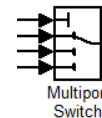
- Vergleicher:
(*Lib: Logic and Bit-Operations*)



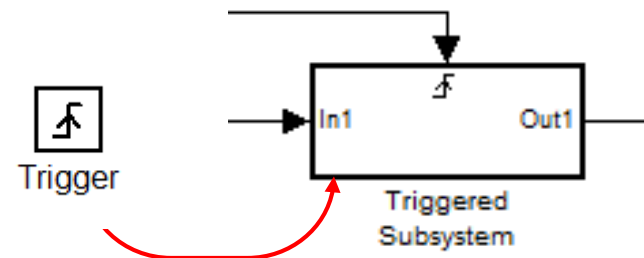
- Schalter mit Schwellwert:
(*Lib: Signal Routing*)



(→ Großer Unterschied zum „Multiportswitch“ ohne Zero-Crossing-Detection)



- Getriggerte Subsysteme:
(*Lib: Ports and Subsystems*)



6. Simulink - 6.4 Simulationsverfahren/-parameter

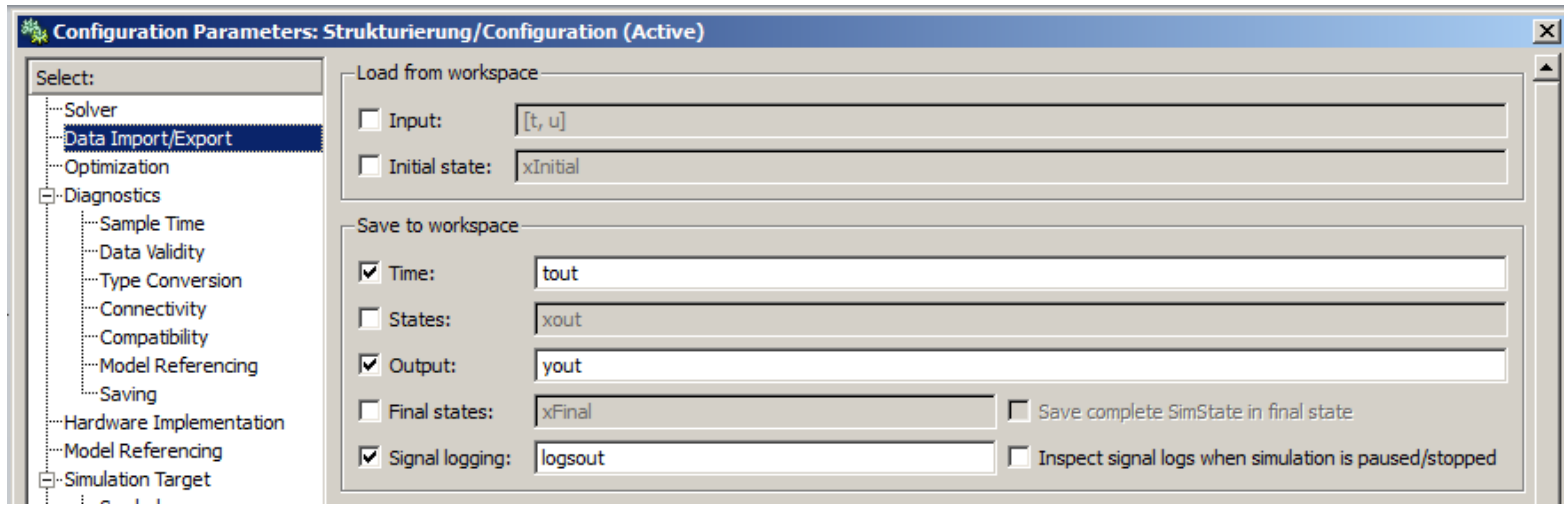
Empfohlene Vorgehensweise für das Festlegen der Simulationsparameter:

Zunächst:

- Variable Step-Size (wenn nicht zwingend anders notwendig)
- Erste Simulation mit default-Einstellungen, dann bei Bedarf Anpassung von:
 - Simulationsalgorithmus,
 - Toleranzen und min./max. Schrittweiten
 - Zero-Crossing Detection
 - gewünschte Genauigkeit
 - usw.

6. Simulink - 6.4 Simulationsverfahren/-parameter

Data Im-/Export:

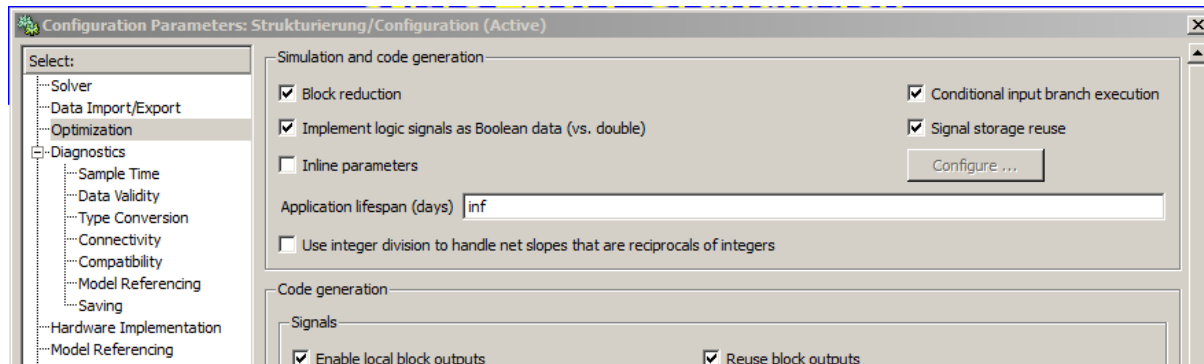


Diverse Simulationsoptionen bzgl. Daten-Im- und Export, z. B.:

- Speichern von Simulationsdaten auf den Matlab-Workspace.
- Speicher von Zustandsvariablen am Simulationsende und
- Neustart der Simulation mit gespeicherten Zustandswerten.
- USW...

6. Simulink - 6.4 Simulationsverfahren/-parameter

Optimierungseinstellungen:

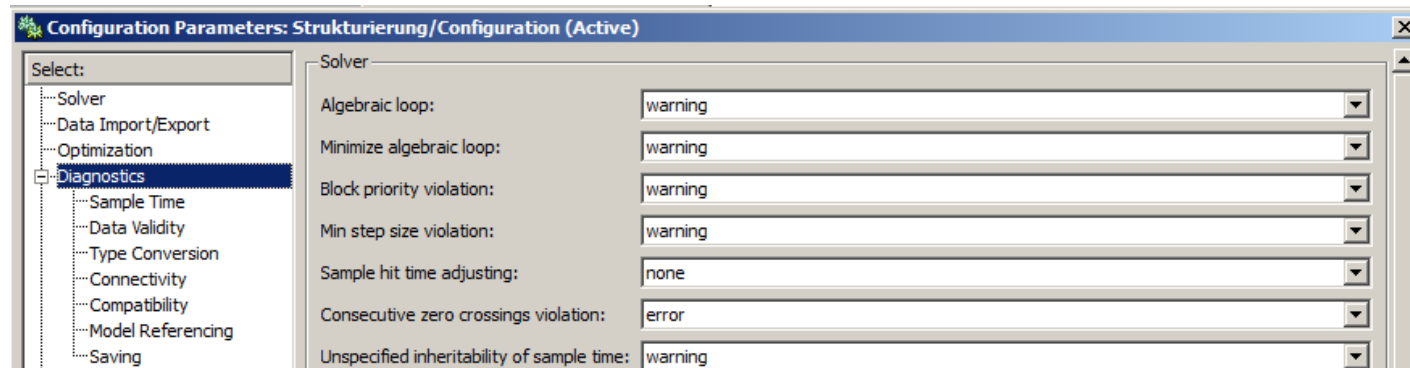


Diverse Simulationsoptionen bzgl. Rechen- und Speicheraufwand, z. B.:

- Umwandlung von Boolean-Signalen zu Double-Signalen (z. B. für Logical Operator)
- Online-Verstellbarkeit von Blockparametern (Parameter inlining)
- Verhalten bei bedingter Bearbeitung

6. Simulink - 6.4 Simulationsverfahren/-parameter

Diagnose:



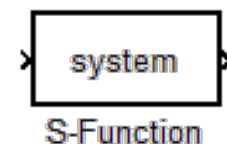
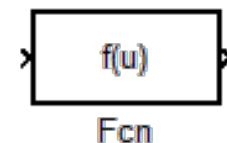
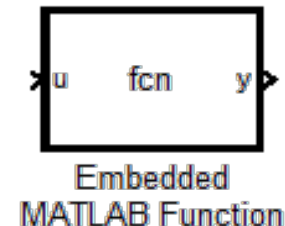
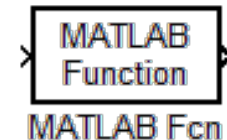
Diverse Simulationsoptionen bzgl. des Verhaltens bei Fehlern und Warnungen:

- Algebraische Schleifen
- Signalinkompatibilitäten
- Schrittweitenbegrenzung
- usw...

6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Benutzerdefinierte Blöcke:

- Blöcke in benutzerdefinierten Bibliotheken
- *Matlab Function*: Aufruf einer internen oder selbstgeschriebenen M-Function (nicht geeignet für Dynamik).
- *Embedded Matlab Function*: Wie *Matlab Function* aber eingebettet im Simulink-Modell.
- *Function (Fcn)*: Einfacher mathematischer Ausdruck in m-Syntax (reduzierter Sprachumfang, keine Dynamik)
- *S-Function*: Selbstdefinierter Block (statisch oder dynamisch, kontinuierlich oder diskret, Zustandsgrößen, usw.)

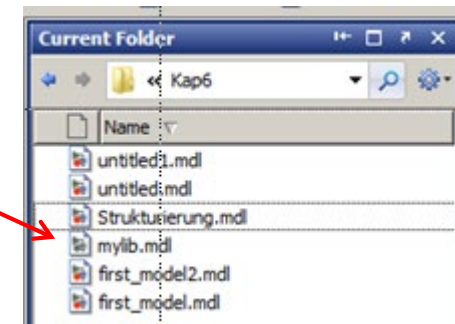
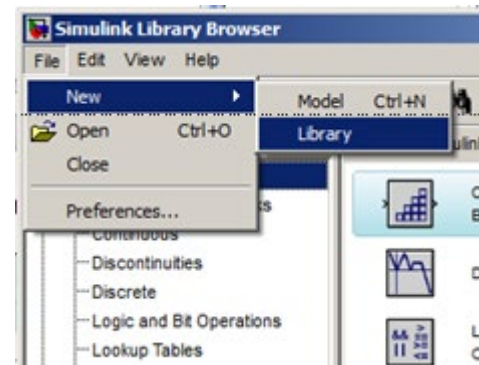


6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Bibliotheken:

In Simulink können sehr einfach benutzerdefinierte Block-Bibliotheken angelegt werden. Diese sind hilfreich wenn identische Funktionen an mehreren Stellen im Modell oder in mehreren Modellen benötigt werden.

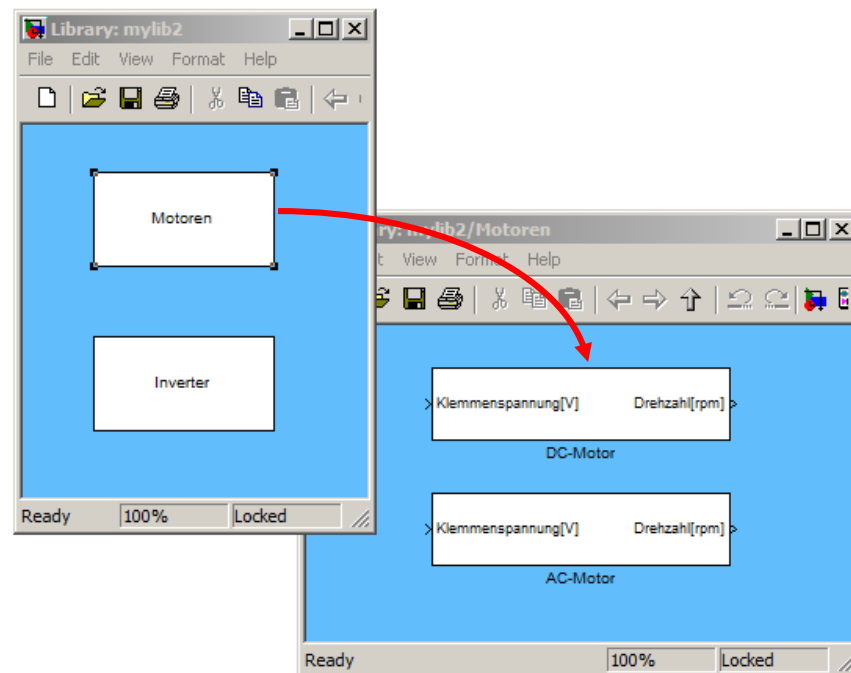
- Erzeugen einer Simulink „Library“
File → New → Library
- Extension: *.mdl
- Unterschiedliche Darstellung im Matlab-Browser
- Bibliotheken können wie Modell bearbeitet werden, sind aber nach dem Schließen zunächst geschützt (automatische Abfrage bei Änderungsversuch).



6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Aufbau von Bibliotheken:

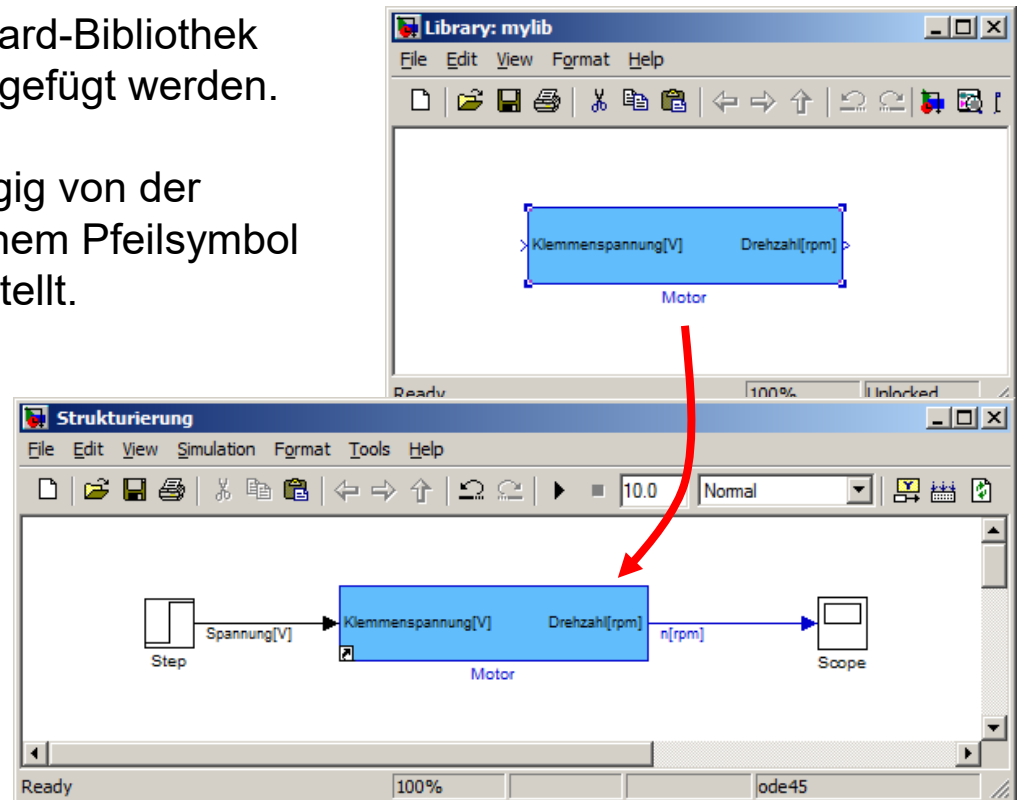
- Die in Modellen nutzbaren Elemente von Simulink-Bibliotheken können einfache Blöcke oder S-Function-Blöcke oder aber auch Subsysteme mit umfangreicher inneren Strukturen sein.
- In Bibliotheken können auch Subsysteme als Gliederungselemente (Subbibliotheken) in beliebiger Tiefe enthalten.



6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Benutzung von Bibliotheken:

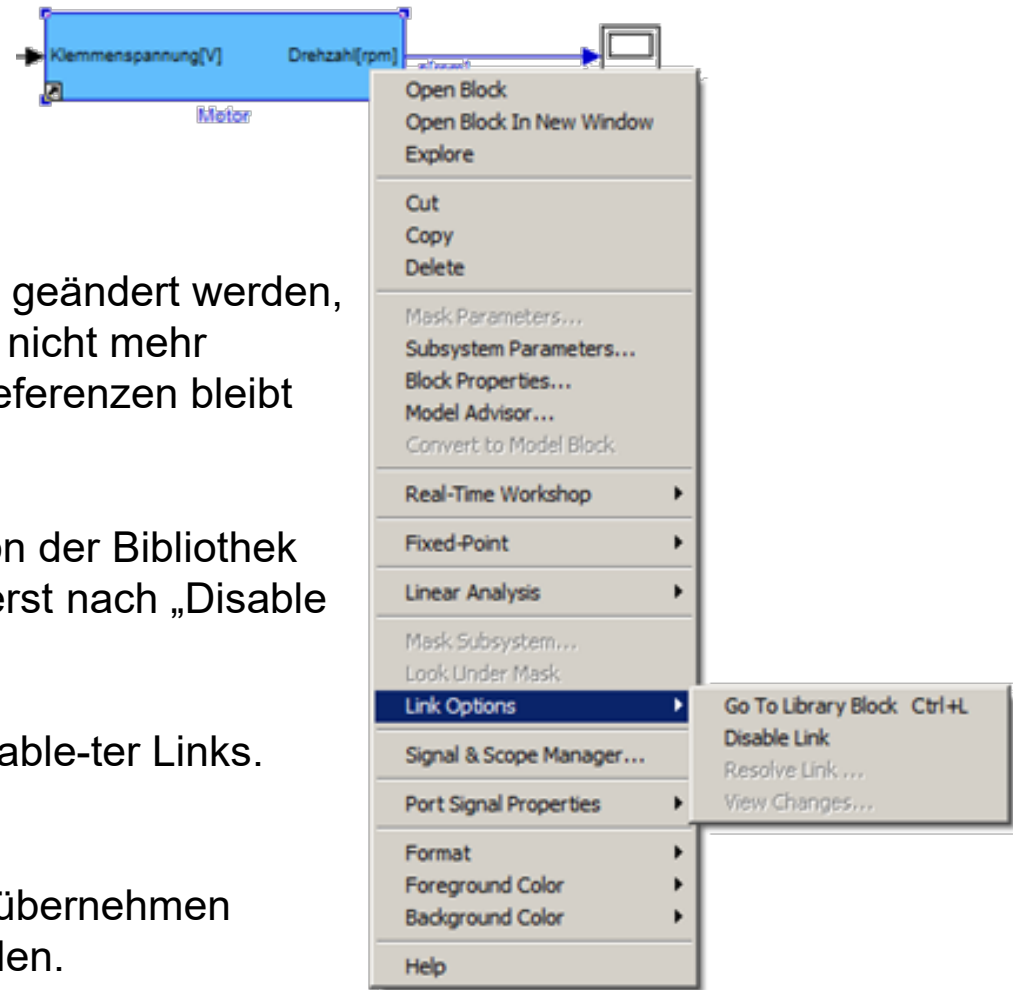
- Blöcke können wie aus der Standard-Bibliothek per „Drag and Drop“ in Modell eingefügt werden.
- Bibliotheksblöcke werden (abhängig von der Einstellung unter „Format“) mit einem Pfeilsymbol in der linken unteren Ecke dargestellt.
- Bibliotheksblöcke sind per Referenz eingebunden, d. h. sie ändern sich wenn die Bibliothek geändert wird.
- Die Referenz ist der Name der Bibliotheksdatei (*.mdl) und der Blockname in der v, d. h. Änderungen führen dazu, dass Bibliothek-Links nicht wiedergefunden werden.
- Benutzte Bibliotheken müssen in den Suchpfaden liegen. Bei gleichnamige Bibliotheken gelten identische Prioritätsregeln wie bei Matlab-Funktionen.



6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Kontext-Menü → Link Options:

- *Go To Library Block*: Anspringen des aktuellen Bibliotheksblocks
- *Disable Link*: Block kann im Modell geändert werden, Änderungen der Bibliothek werden nicht mehr automatisch übernommen. Die Referenzen bleibt aber erhalten
- *Break Link*: Block wird endgültig von der Bibliothek getrennt. Referenz geht verloren (erst nach „Disable Link“ möglich).
- *Resolve Link*: Wiederherstellen disable-ter Links.
Optionen:
 - Änderungen aus in Bibliothek übernehmen
 - Bibliotheksblock wiederherstellen.



6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Hinweise zu Bibliotheken:

- Das Arbeiten mit Bibliotheken erfordert wie bei Funktionen eine große Disziplin bei Änderungen, Verlinken, „Disablen“, Propertgieren, usw..
- Benutzer definierte Bibliotheken möglichst in abgeschlossenen Projekten halten.
- „Globale“ Bibliotheken (außerhalb eines abgeschlossenen Projekts) vermeiden oder nur als Bestandteil von zentral gepflegten Tools verwenden.

Achtung:

- Bibliotheken können selber Bibliotheksblöcke enthalten.
Solche Konstruktionen sollte (mit wenigen sinnvollen Ausnahmen) nach Möglichkeit vermieden werden.

6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

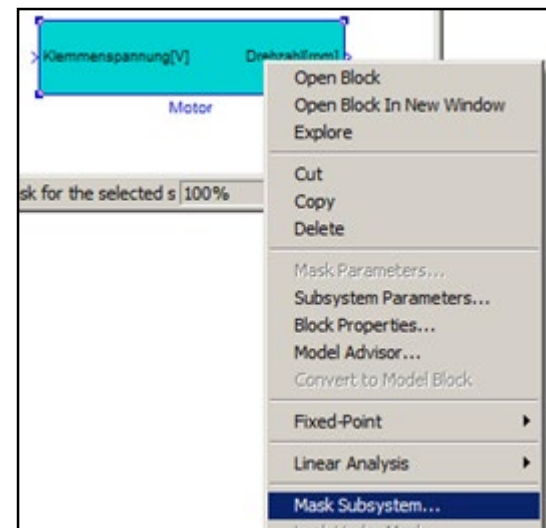
Maskierte Blöcke und Subsysteme:

Das „Maskieren“ von Blöcken und Subsystemen ermöglicht es eigene Blöcke (in Bibliotheken oder direkt im Modell) benutzerfreundlich zu gestalten. Dies beinhaltet:

- Individuelle Parametrierungsmaske,
- Individuelles Layout
- Unterlagerte Initialisierungsfunktionen
- Beschreibungs- und Hilfetext
- Call-Back-Funktion

Maskieren eines Subsystems:

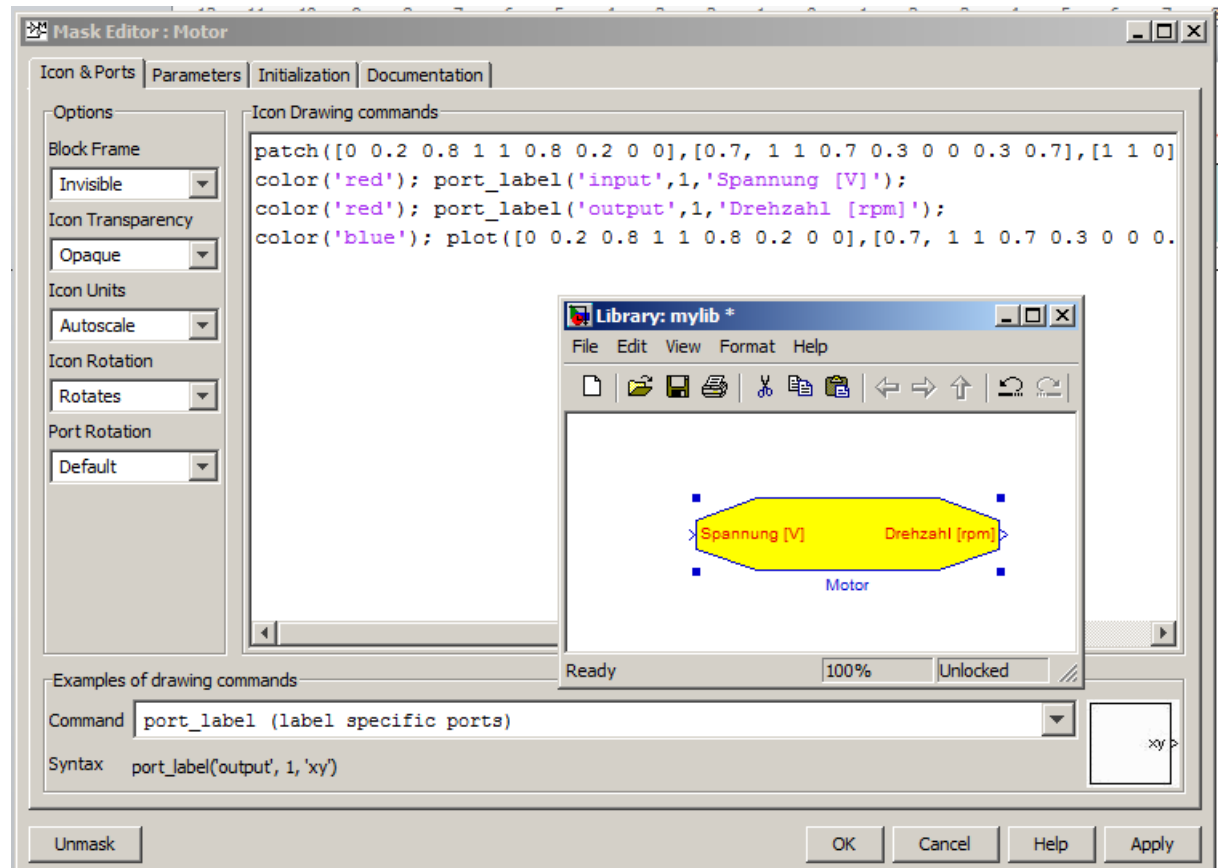
- Kontextmenü → Mask Subsystem
- Funktioniert auch bei div. Einzelblöcken (z. B. s-Function)



6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

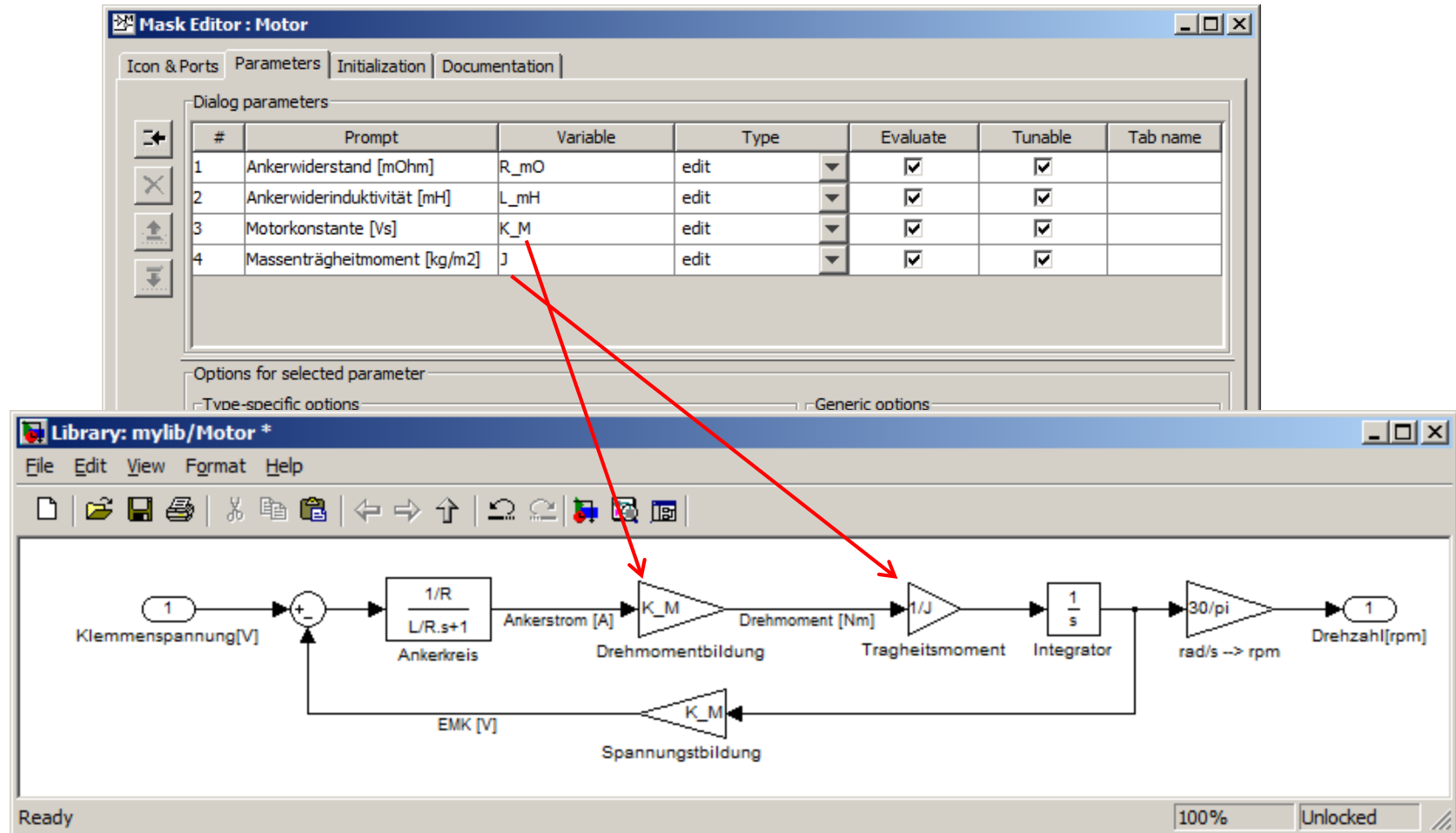
Individuelles BlockLayout: (Icons & Ports)

- Blockgestaltung mit begrenztem Befehlssatz (s. Command)
- Auch in Abhängigkeit von Parametern oder gesteuert durch Initialisierungsfunktion möglich



6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Maskenparameter: (Parameter)



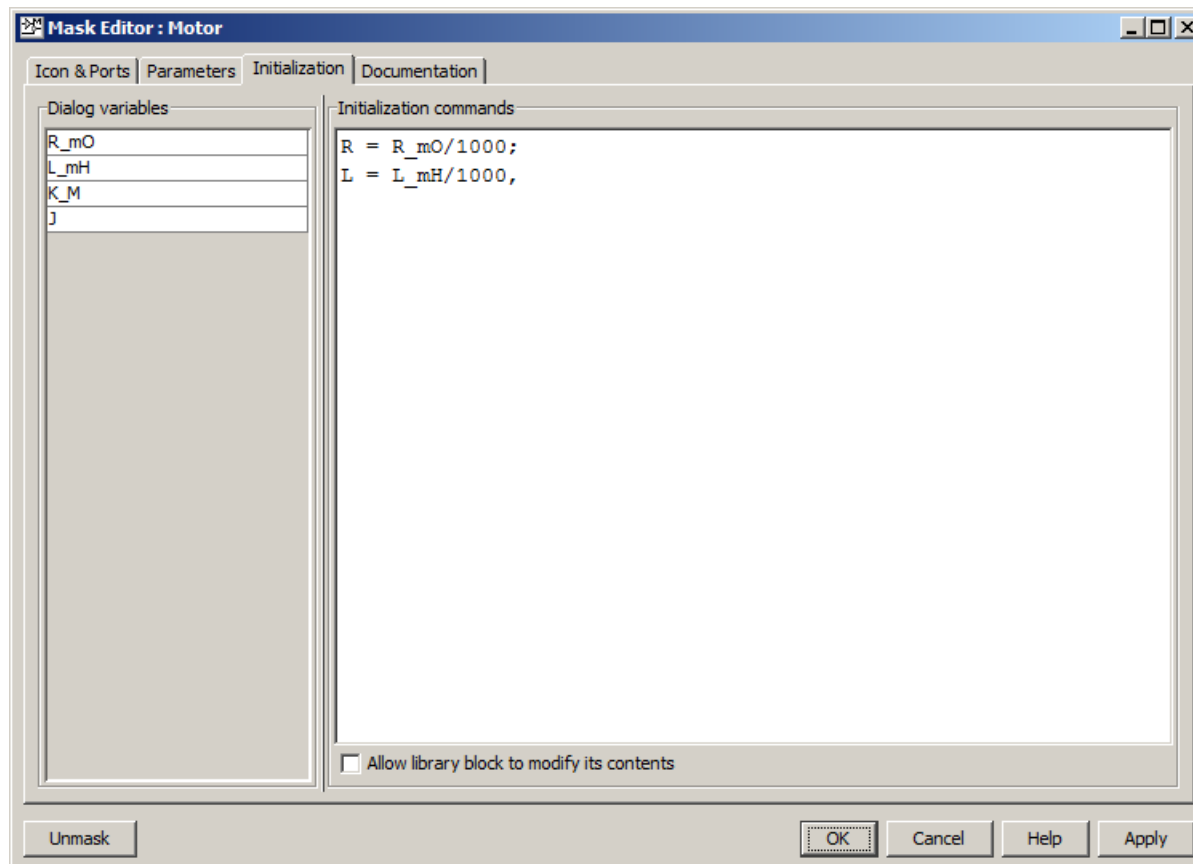
6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

- Maskenparameter sind nur für die Modellteile unter der Maske sichtbar (vgl. Funktionen). Workspace-Parameter sind aber auch weiterhin unter Masken verfügbar sofern nicht ein gleichnamiger Maskenparameter existiert.
Regel: Besser alle Parameter unter einer Maske als MaskenParameter verwalten.
- Als Eingabeelemente stehen auch Checkboxen oder Popup-Menüs zur Verfügung.
- Masken-Parameter können auch in Call-Backs oder in der Initialisierungsfunktion verwendet werden

6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Initialisierung: (Initialization)

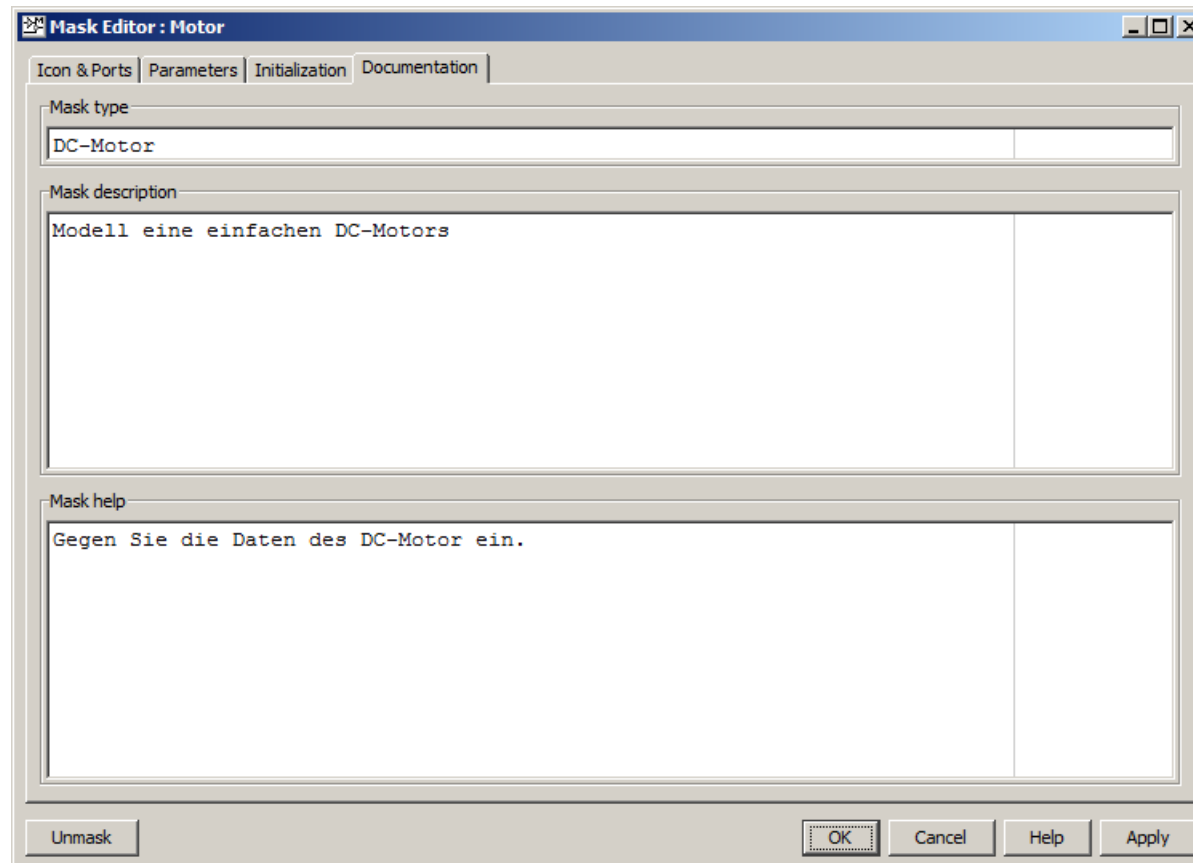
- Die Initialisierungsseite ermöglicht die Angabe diverser Initialisierungs-kommandos für den Block (u. U. auch aufwendig in Externen Funktionen).



6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

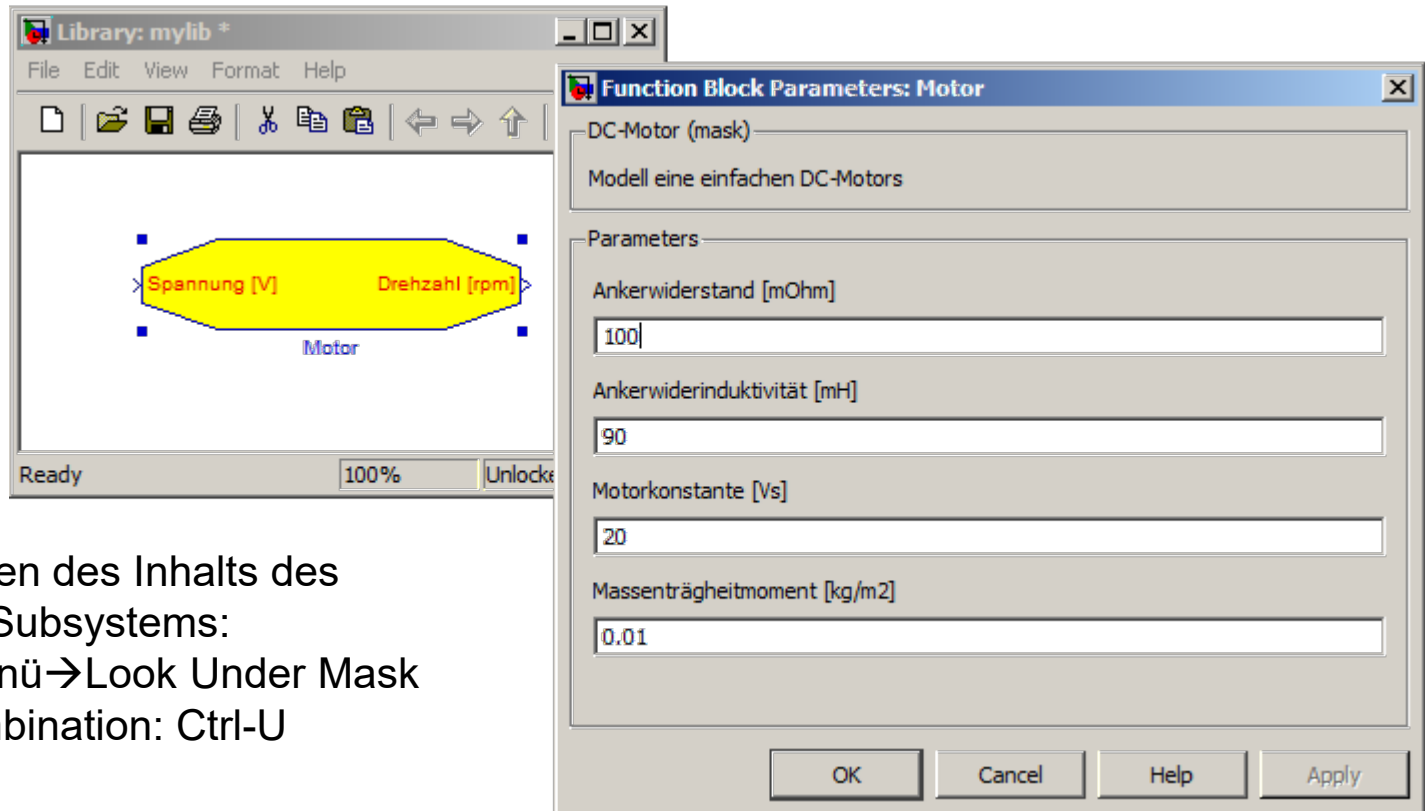
Doku und Hilfe: (Documentation)

- Die Dokumentierungsseite ermöglicht die Angabe von Doku- und Hilfetext der in der Maske angezeigt wird.



6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Öffnen der Maske durch Doppelklick:



- Zum Anzeigen des Inhalts des maskierten Subsystems:
 - Kontextmenü → Look Under Mask
 - Tastenkombination: Ctrl-U

6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

S-Funktions:

- S-Function können in „M“ oder in „C“ geschrieben werden und ermöglichen das Erstellen von sehr mächtigen benutzer-definiert Blöcke unter Nutzung der vollen Leistungsfähigkeit und aller Optionen des Simulations-Kernals von Simulink.
- S-Function basieren auf weitgehend vorgegebenen Funktionen und Datenstrukturen, die es dem Anwender ermöglichen die Eigenschaften und das Verhalten eines Blocks zu definieren, wie:
 - Anzahl und Datentypen der Parameter
 - Anzahl, Vektorbreiten, Datentypen, und Verhalten von Ein- und Ausgängen.
 - Anzahl kontinuierlichen und diskreten Zustandgrößen
 - Abtastzeiten, direkter Durchgriff
 - Inertialzustand
 - Blockverhalten:
 - Ausgangswerte
 - Ableitung bei kontinuierlichen Zustandgrößen
 - Zustand $n+1$) bei diskreten Zustandgrößen

6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

Beispiel s-Function

```
/* File      : csfunc.c
 * Abstract:
 *
 *      Example C-file S-function for defining a continuous system.
 *
 *       $x' = Ax + Bu$ 
 *       $y = Cx + Du$ 
 *
 *      For more details about S-functions, see simulink/src/sfuntmpl_doc.c.
 *
 *      Copyright 1990-2009 The MathWorks, Inc.
 *      $Revision: 1.1.6.1 $
 */

#define S_FUNCTION_NAME csfunc
#define S_FUNCTION_LEVEL 2

#include "simstruc.h"

#define U(element) (*uPtrs[element]) /* Pointer to Input Port0 */
```

6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

```
static real_T A[2][2]={ { -0.09, -0.01 } ,  
                        { 1      , 0      }  
                        };  
  
static real_T B[2][2]={ { 1      , -7      } ,  
                        { 0      , -2      }  
                        };  
  
static real_T C[2][2]={ { 0      , 2      } ,  
                        { 1      , -5      }  
                        };  
  
static real_T D[2][2]={ { -3      , 0      } ,  
                        { 1      , 0      }  
                        };
```


6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

```
/*=====*
 * S-function methods *
 *=====*/

/* Function: mdlInitializeSizes =====
 * Abstract:
 *   The sizes information is used by Simulink to determine the S-function
 *   block's characteristics (number of inputs, outputs, states, etc.).
 */
static void mdlInitializeSizes(SimStruct *S)
{
    ssSetNumSFcnParams(S, 0); /* Number of expected parameters */
    if (ssGetNumSFcnParams(S) != ssGetSFcnParamsCount(S)) {
        return; /* Parameter mismatch will be reported by Simulink */
    }

    ssSetNumContStates(S, 2);
    ssSetNumDiscStates(S, 0);

    if (!ssSetNumInputPorts(S, 1)) return;
    ssSetInputPortWidth(S, 0, 2);
    ssSetInputPortDirectFeedThrough(S, 0, 1);

    if (!ssSetNumOutputPorts(S, 1)) return;
    ssSetOutputPortWidth(S, 0, 2);

    ssSetNumSampleTimes(S, 1);
    ssSetNumRWork(S, 0);
    ssSetNumIWork(S, 0);
    ssSetNumPWork(S, 0);
    ssSetNumModes(S, 0);
    ssSetNumNonsampledZCs(S, 0);
    ssSetSimStateCompliance(S, USE_DEFAULT_SIM_STATE);
}
```

6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

```
/* Take care when specifying exception free code - see sfuntmpl_doc.c */
ssSetOptions(S, SS_OPTION_EXCEPTION_FREE_CODE);
}

/* Function: mdlInitializeSampleTimes =====
 * Abstract:
 *   Specify that we have a continuous sample time.
 */
static void mdlInitializeSampleTimes(SimStruct *S)
{
    ssSetSampleTime(S, 0, CONTINUOUS_SAMPLE_TIME);
    ssSetOffsetTime(S, 0, 0.0);
    ssSetModelReferenceSampleTimeDefaultInheritance(S);
}

#define MDL_INITIALIZE_CONDITIONS
/* Function: mdlInitializeConditions =====
 * Abstract:
 *   Initialize both continuous states to zero.
 */
static void mdlInitializeConditions(SimStruct *S)
{
    real_T *x0 = ssGetContStates(S);
    int_T lp;

    for (lp=0;lp<2;lp++) {
        *x0++=0.0;
    }
}
```

6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

```

/* Function: mdlOutputs =====
* Abstract:
*      y = Cx + Du
*/
static void mdlOutputs(SimStruct *S, int_T tid)
{
    real_T      *y      = ssGetOutputPortRealSignal(S,0);
    real_T      *x      = ssGetContStates(S);
    InputRealPtrsType uPtrs = ssGetInputPortRealSignalPtrs(S,0);

    UNUSED_ARG(tid); /* not used in single tasking mode */

    /* y=Cx+Du */
    y[0]=C[0][0]*x[0]+C[0][1]*x[1]+D[0][0]*U(0)+D[0][1]*U(1);
    y[1]=C[1][0]*x[0]+C[1][1]*x[1]+D[1][0]*U(0)+D[1][1]*U(1);
}

#define MDL_DERIVATIVES
/* Function: mdlDerivatives =====
* Abstract:
*      xdot = Ax + Bu
*/
static void mdlDerivatives(SimStruct *S)
{
    real_T      *dx      = ssGetdX(S);
    real_T      *x      = ssGetContStates(S);
    InputRealPtrsType uPtrs = ssGetInputPortRealSignalPtrs(S,0);

    /* xdot=Ax+Bu */
    dx[0]=A[0][0]*x[0]+A[0][1]*x[1]+B[0][0]*U(0)+B[0][1]*U(1);
    dx[1]=A[1][0]*x[0]+A[1][1]*x[1]+B[1][0]*U(0)+B[1][1]*U(1);
}

```

6. Simulink - 6.5 Benutzerdefinierte Blöcke und Bibliotheken

```
/* Function: mdlTerminate =====
 * Abstract:
 *   No termination needed, but we are required to have this routine.
 */
static void mdlTerminate(SimStruct *S)
{
    UNUSED_ARG(S); /* unused input argument */
}

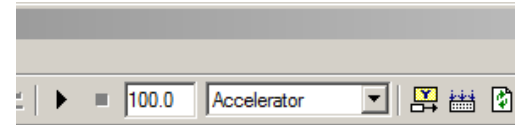
#ifdef MATLAB_MEX_FILE    /* Is this file being compiled as a MEX-file? */
#include "simulink.c"      /* MEX-file interface mechanism */
#else
#include "cg_sfun.h"       /* Code generation registration function */
#endif
```

6. Simulink - 6.6 Besonderheiten

Alternative Simulations-Modi:

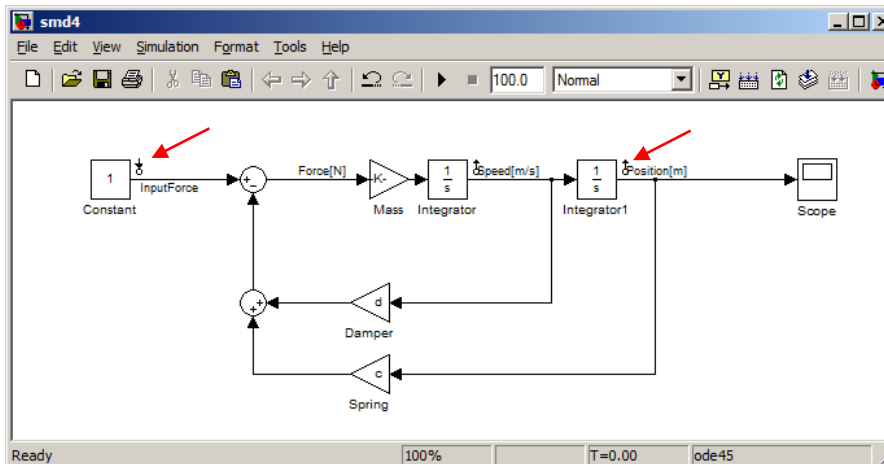
1. Beschleunigte Simulation: „Accelerator“-Mode:

- Modell wird vorkompiliert.
- Zusätzliche Zeit für das Vorkompilieren (nicht sinnvoll bei kurzer Simulationszeit).
- Parameter sind nicht mehr zur Laufzeit änderbar.



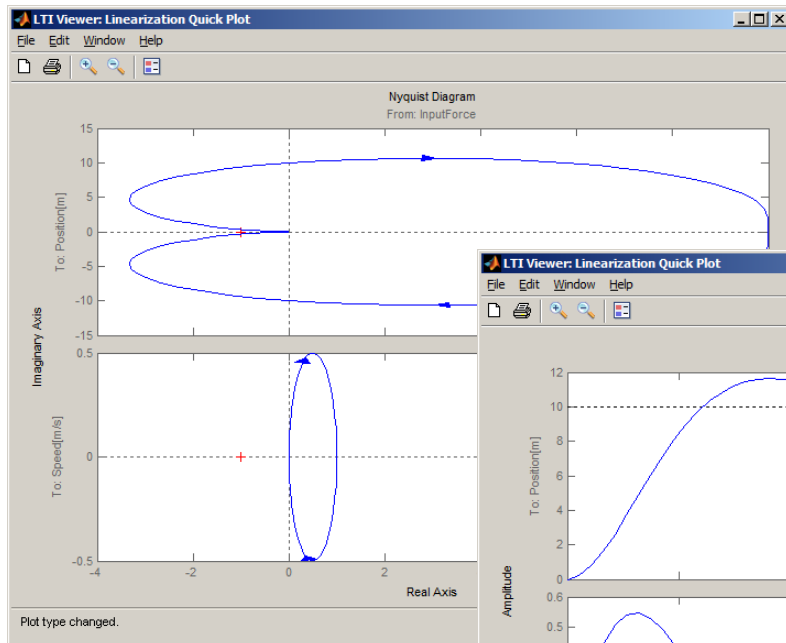
2. Linearisierung:

- „Liniarization Points“ setzen
- „Control and Estimaton Tool Manager“

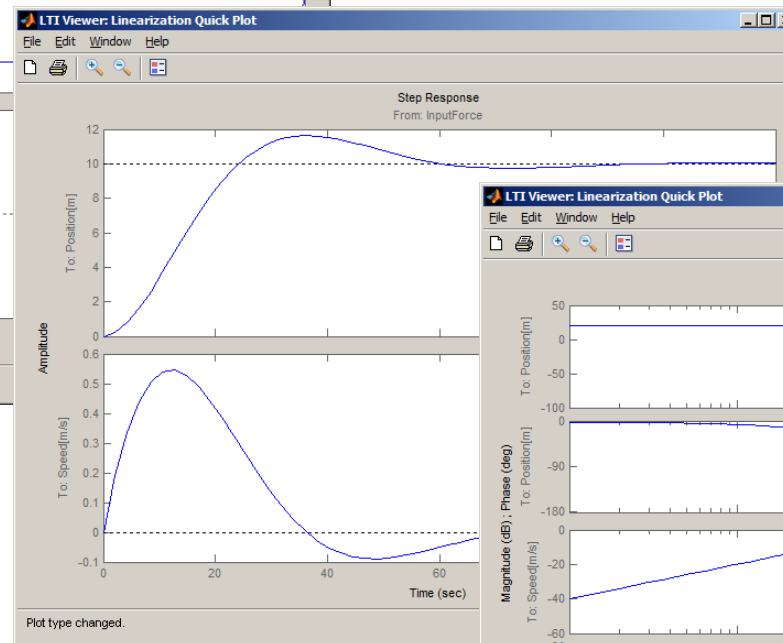


Active	Block Name	Output Port	Signal Name	Configuration	Open Loop
☒	smd4/Constant	1	InputForce	Input	☐
☒	smd4/Integrator 1	1	Position[m]	Output	☐
☒	smd4/Integrator	1	Speed[m/s]	Output	☐

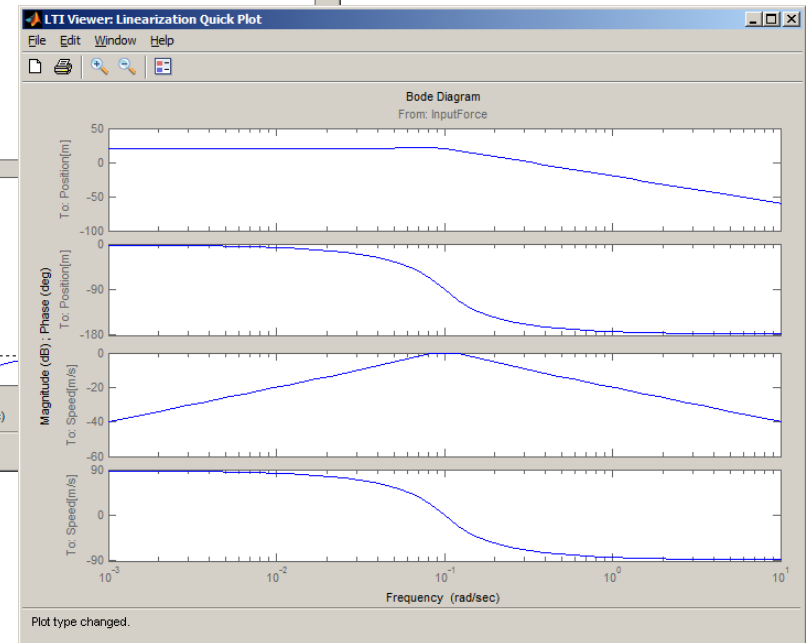
6. Simulink - 6.6 Besonderheiten



Nyquist-Ortskurve



Sprungantwort

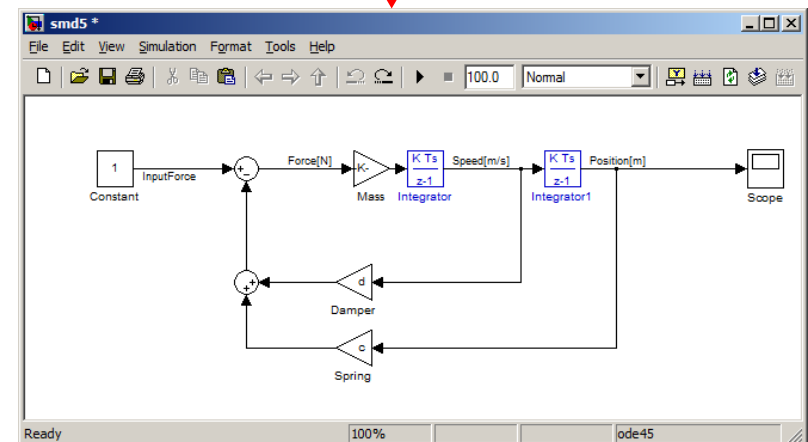
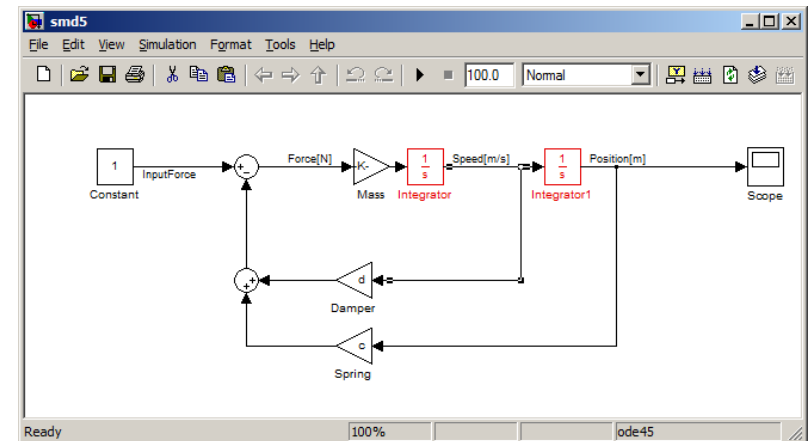
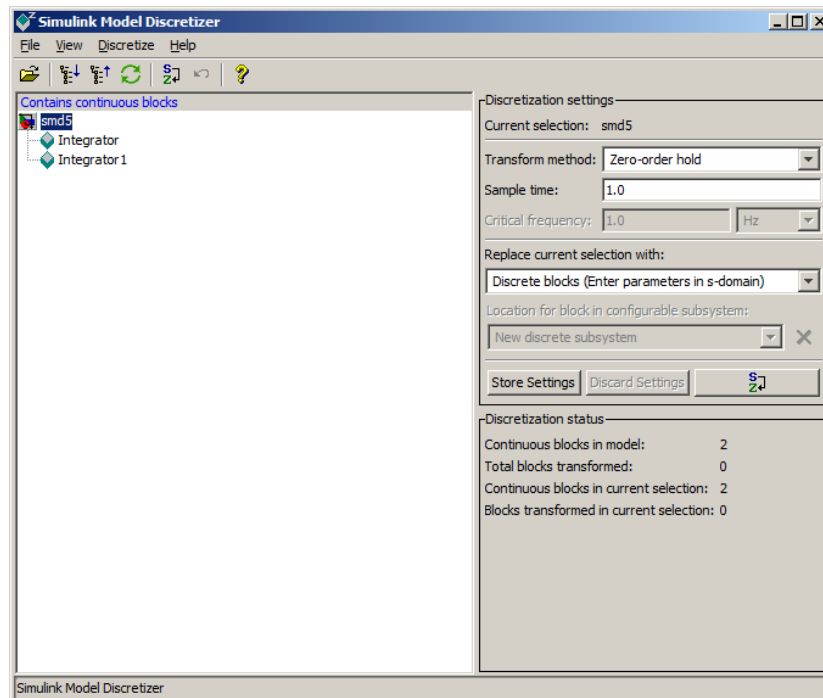


Bode-Diagramm

6. Simulink - 6.6 Besonderheiten

Automatische Modelldiskretisierung:

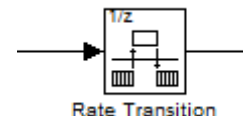
- Model Discretizer



6. Simulink - 6.6 Besonderheiten

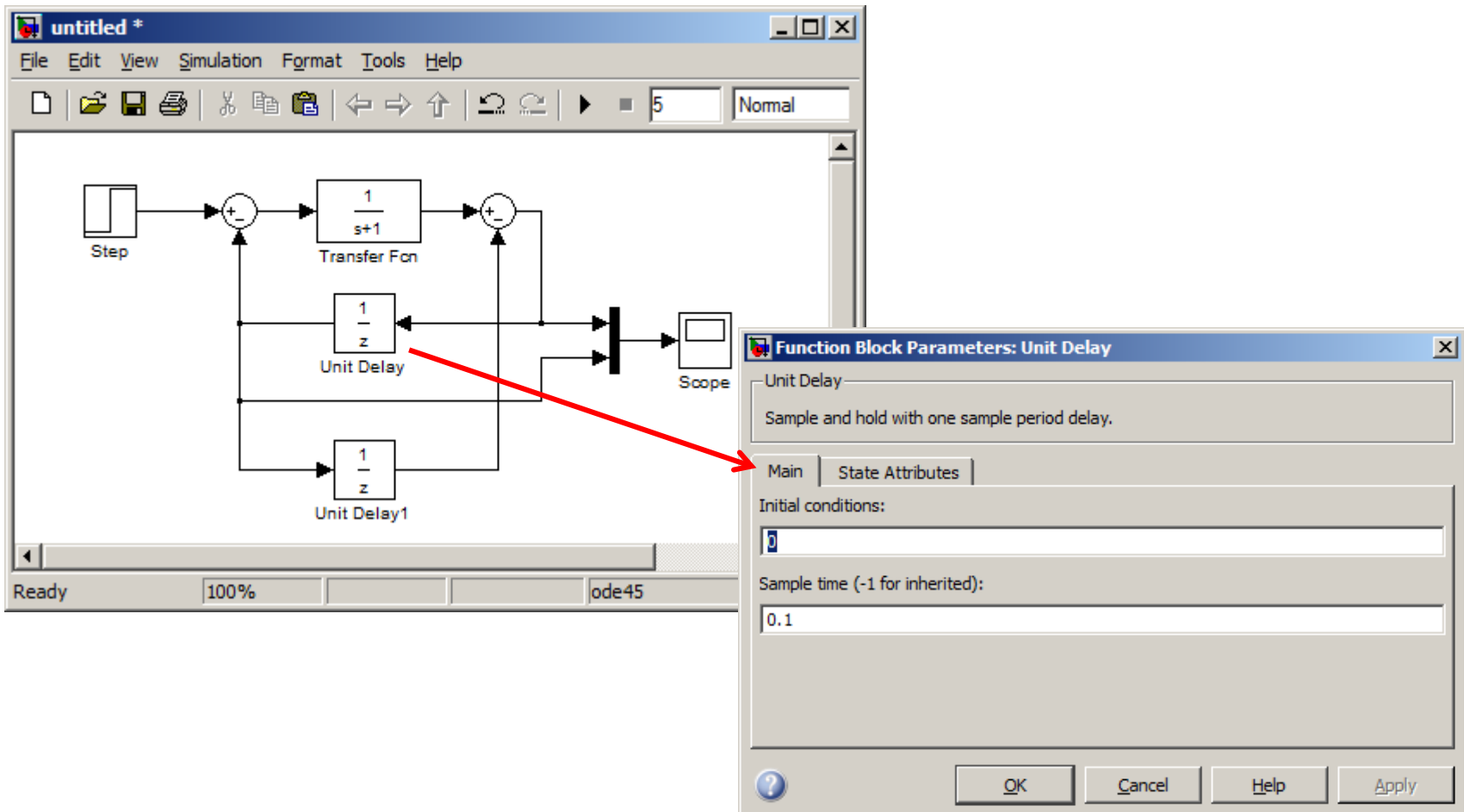
Abtastzeiten:

- Blöcke für kontinuierliche Übertragungsfunktionen müssen mit der Basis-Abtastzeit (ggf. variabel) des Modells ausgeführt werden. Da der Simulationsalgorithmus die Abtastzeit kennen muss, können Blöcke für kontinuierliche Übertragungsfunktionen also nicht in getriggerten oder sog. „function-called“-Subsysteme ausgeführt werden.
- Diskrete Übertragungsfunktionen und viele andere (nicht dynamische) Blöcke kann die Abtastzeit unabhängig von der Modellabtastzeit angegeben werden. Dies ist für die Darstellung von Abtastsystemen auch zwingend notwendig.
- Die sog. „Inherent Sample-Time“ (oft durch ‚-1‘ indiziert) bewirkt für den Block die Übernahme der Abtastzeit vom Zielblock (Back Propagation) bzw. die Verwendung der Modellabtastzeit oder der in einem „nicht-virtuellen“ Subsystem gültigen Abtastzeit, d.h. die Ausführung des Blocks bei jeder Ausführung des Subsystems.
- Unter echtzeitbedingungen muss beim Übergang zwischen Modellteilen mit unterschiedlicher Abtastzeit zusätzlich die Datenkonsistenz sichergestellt werden. Hierzu dient der „Rate-Transition“-Block:



6. Simulink - 6.6 Besonderheiten

Beispiel für unterschiedliche Abtastzeiten:



6. Simulink - 6.7 Steuern von Simulink aus Matlab

Steuern von Simulink aus Matlab:

Simulink kann aus Matlab (d. h. über m-Skripte) gesteuert werden. Es gibt Befehle zur

- Steuerung der Simulation
- Manipulation von Blockparameter
- Aufbau und Änderung von Modellen

Zweck der Steuerung aus Matlab:

- Automatische Steuerung von Serienexperimenten (z. B. Simulation eines Systems mit unterschiedlichen Parametern).
- Automatische komplexe Konfiguration von Subsystem (z. B. ein Subsystem, dass sich abhängig von der Parametrierung intern strukturelle umkonfiguriert).
- Automatische Erzeugung von Modellen oder Teilmodellen (z. B. Erzeugung von Modellteilen mit weitgehend generischer Struktur die Mittels anderen Quellendaten – z. B. Excellisten - spezifiziert werden können).

6. Simulink - 6.7 Steuern von Simulink aus Matlab

Steuern der Simulation:

Der Befehl `sim(...)`:

Allgemein:

```
simOut = sim('model', 'ParameterName1', Value1, ...);
```

Beispiel 1: Starten des Modells „model“ mit Default-Parametern

```
simOut = sim('model');
```

Beispiel 2: Starten des Modells „model“ mit ode45 und 5 Sekunden Simulationszeit.

```
simOut = sim('model', 'SolverName', 'ode45', 'StopTime', '5');
```

6. Simulink - 6.7 Steuern von Simulink aus Matlab

Alternative: Konfiguration der Simulationsparameter über einen Parameter-Strukt:

```
paramNameValStruct.SimulationMode = 'rapid';  
paramNameValStruct.AbsTol         = '1e-5';  
paramNameValStruct.SaveState      = 'on';  
paramNameValStruct.StateSaveName  = 'xoutNew';  
paramNameValStruct.SaveOutput     = 'on';  
paramNameValStruct.OutputSaveName = 'youtNew';  
simOut = sim('vdp',paramNameValStruct);
```

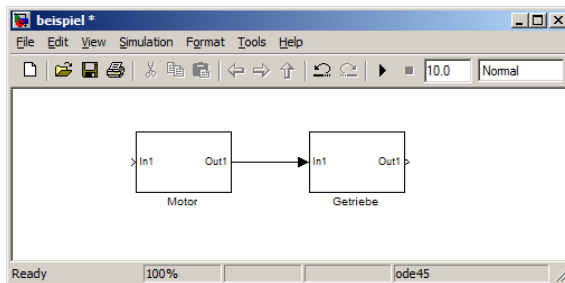
6. Simulink - 6.7 Steuern von Simulink aus Matlab

Weitere wichtige Befehle:

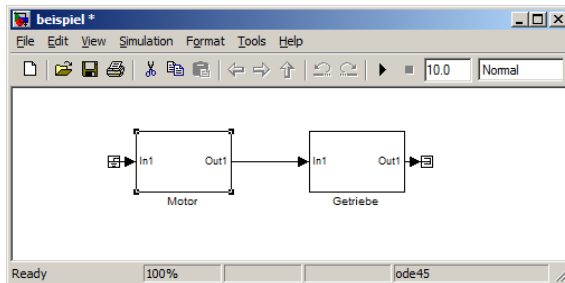
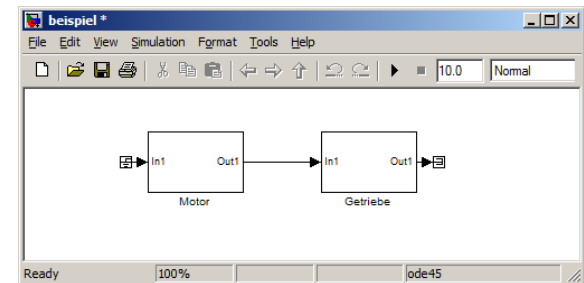
<code>gcb</code>	Modellpfad des aktuellen Blocks
<code>gcbh</code>	Handle des aktuellen Blocks
<code>gcs</code>	Modellpfad des aktuellen Systems (Modell oder Subsystem)
<code>bdroot</code>	Name des aktuellen Modells
<code>get_param</code>	Auslesen von Block-, Modell- oder Systemparametern
<code>set_param</code>	Schreiben von Block-, Modell- oder Systemparametern
<code>add_block</code>	Hinzufügen eine Blocks
<code>add_line</code>	Hinzufügen einer Signalline
<code>addterms</code>	Hinzufügen von Terminatoren (Einziger Befehl für den tägliche Gebrauch)
<code>open_system</code>	Öffnen eines Modells, Systems oder Dialogs
<code>close_system</code>	Schließen eines Modells, Systems oder Dialogs
<code>save_system</code>	Speichern eines Modells

6. Simulink - 6.7 Steuern von Simulink aus Matlab

Beispiel 1:



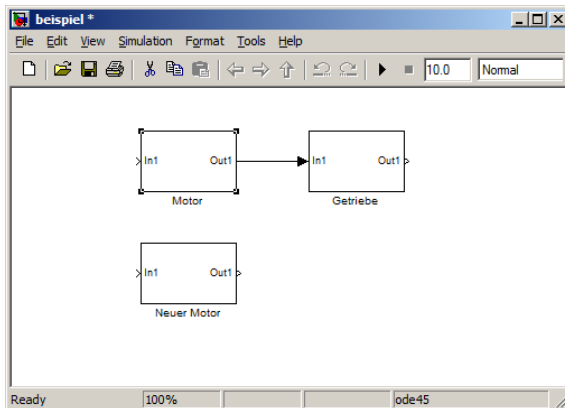
>> addterms(gcs)



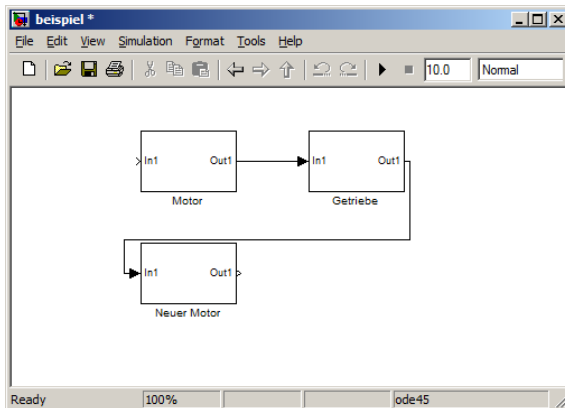
```
>> gcb
ans =
beispiel/Motor
>> gcs
ans =
beispiel
>> gcbh
ans =
3.7340e+003
```

6. Simulink - 6.7 Steuern von Simulink aus Matlab

Beispiel 2:



```
>> handle1 = gcb;  
>> handle2 = add_block(handle1,...  
    [gcs,'/Neuer Motor']);  
>> set_param(handle2,'Position',...  
    get_param(handle1,'Position')...  
    +[0 100 0 100]
```



```
>> add_line(gcs,'Getriebe/1',...  
    'Neuer Motor/1','autorouting','on')
```

6. Simulink - 6.7 Steuern von Simulink aus Matlab

Beispiel 3:

```
>> get_param(gcb, 'ObjectParameters')
```

```
ans =
```

```
        Name: [1x1 struct]  
        Tag: [1x1 struct]  
    Description: [1x1 struct]  
        Type: [1x1 struct]  
        Parent: [1x1 struct]  
        Handle: [1x1 struct]  
    HiliteAncestors: [1x1 struct]  
    RequirementInfo: [1x1 struct]  
        Ports: [1x1 struct]  
        Position: [1x1 struct]  
    Orientation: [1x1 struct]  
    PortRotationType: [1x1 struct]  
    ForegroundColor: [1x1 struct]  
    BackgroundColor: [1x1 struct]  
        DropShadow: [1x1 struct]  
        IOType: [1x1 struct]  
    NamePlacement: [1x1 struct]  
        ShowName: [1x1 struct]  
        Priority: [1x1 struct]  
    AttributesFormatString: [1x1 struct]  
    InstantiateOnLoad: [1x1 struct]  
        AncestorBlock: [1x1 struct]  
        ReferenceBlock: [1x1 struct]  
        LibraryVersion: [1x1 struct]  
    UserDataPersistent: [1x1 struct]  
  
    ...
```


6. Simulink - 6.8 Zusatzbibliotheken

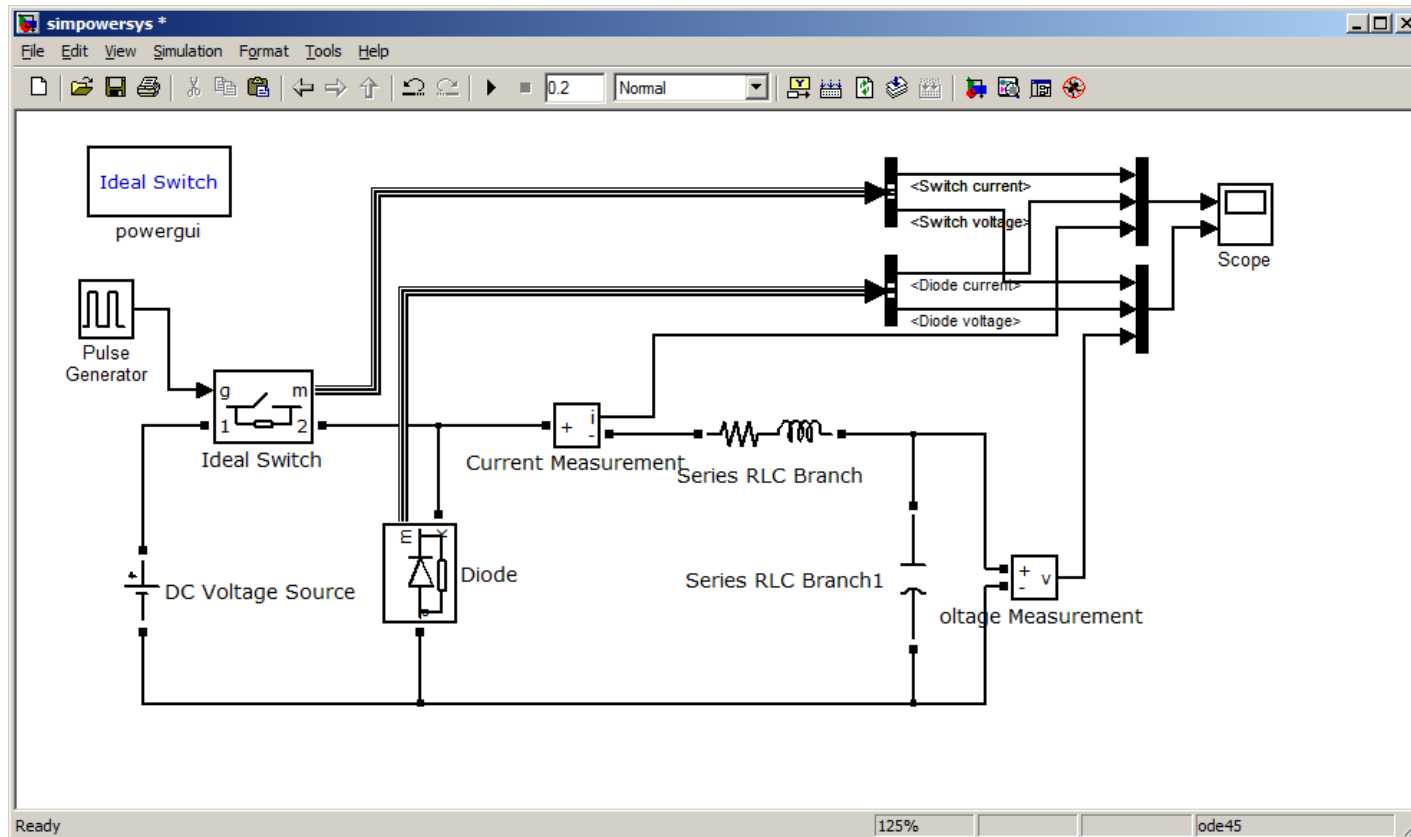
Simulink-Zusatzbibliotheken (Toolboxen):

- *Control System Toolbox* Im wesentliche nur Matlab-Funktionen
- *Fuzzy Logic Toolbox* Im wesentliche nur Matlab-Funktionen
- *Neural Network Toolbox* Blöcke für Neuronale Netze
- ***Real-time Workshop*** C-Code Generierung für Echtzeit-Anwendungen
- ***SimPowerSystems*** Schaltungssimulation
- *SimScape* Mechanische und elektrische Systeme (DAEs)
- ***Stateflow*** Zustandsgraphen

6. Simulink - 6.8 Zusatzbibliotheken

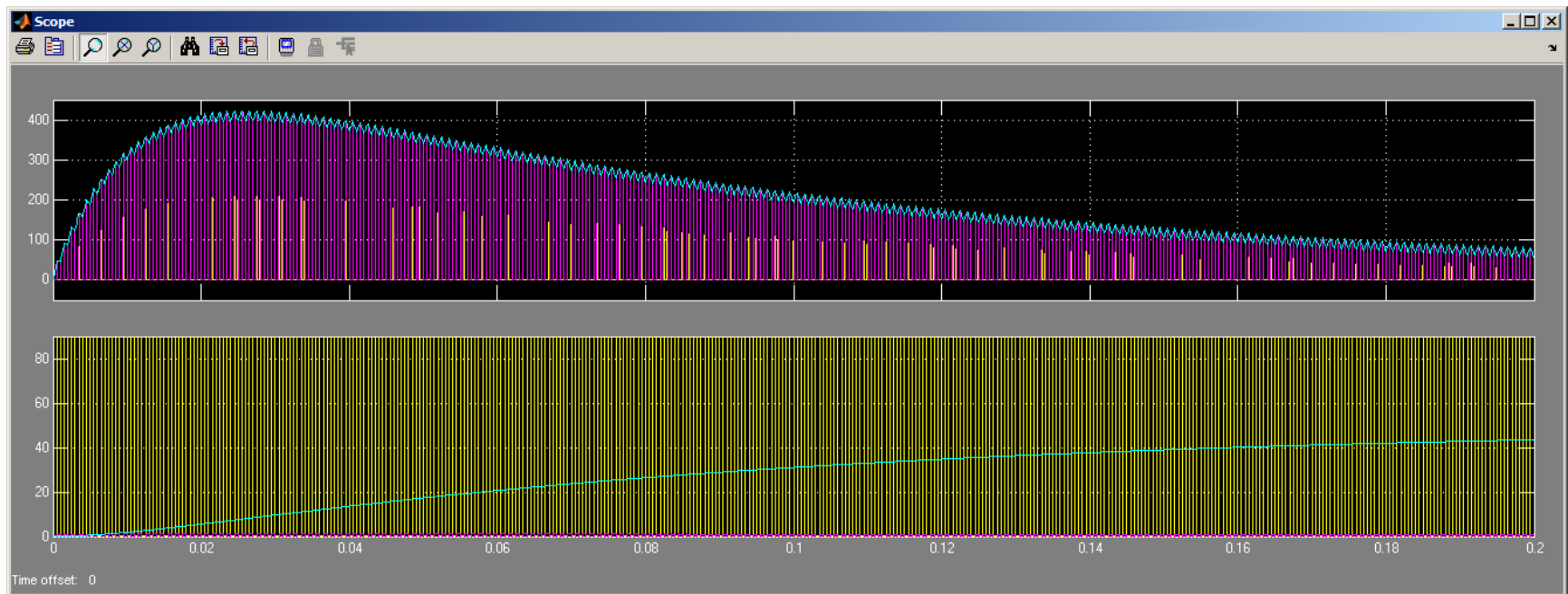
SimPowerSystems:

Schaltungssimulation unter Simulink:



6. Simulink - 6.8 Zusatzbibliotheken

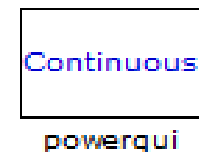
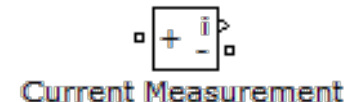
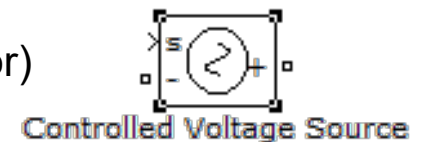
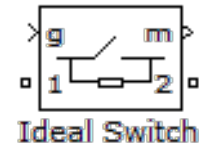
Beispiel: Tiefsetzsteller:



6. Simulink - 6.8 Zusatzbibliotheken

Arbeiten mit SimPowerSystems

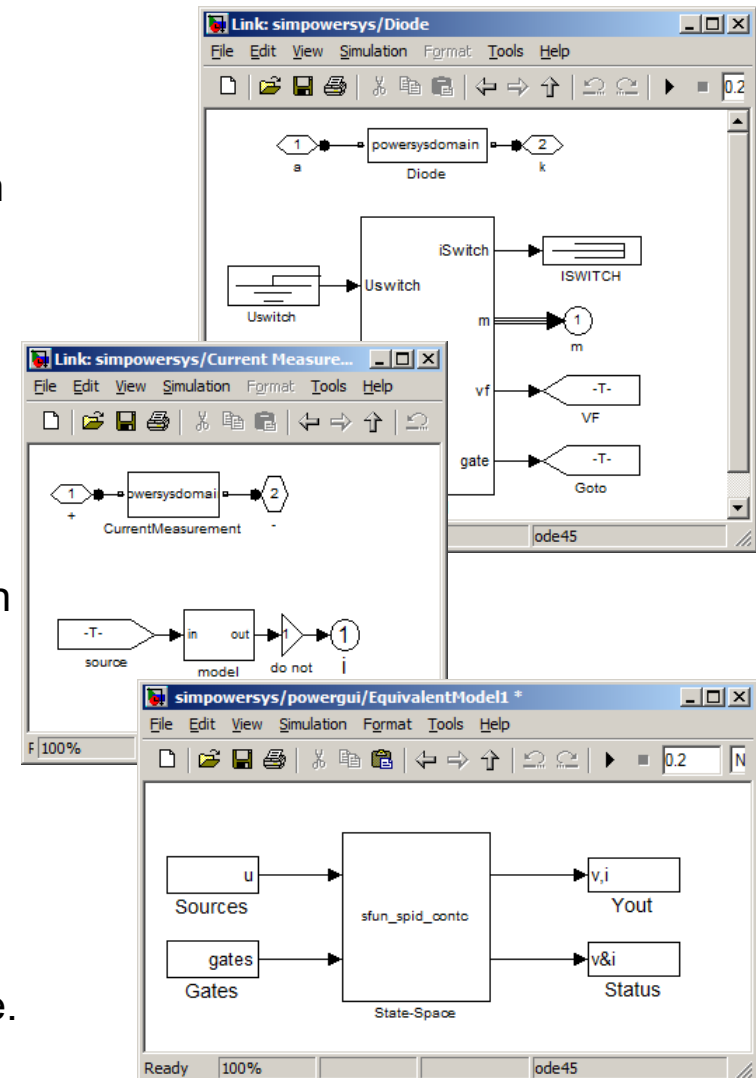
- Elektrische Signalpfade und deren Ports unterscheiden sich von „normalen“ Simulink-Signalen und deren Ports.
- Einige SimPowerSystems-Blöcke haben sowohl Ein- und Ausgänge für Simulink-Signale als auch elektrische „Anschlüsse“.
- Schalter (Transistoren und Strom-/Spannungsquellen können auf diese Weise über Standardsignalquellen (z. B. Signalgenerator) gesteuert werden und „Messblöcke“ können elektrische Simulationsgrößen als weiter verarbeitbare Simulink-Signale ausgeben.
- Der „PowerGUI“ Block dient zur Konfiguration der Simulation.



6. Simulink - 6.8 Zusatzbibliotheken

Funktionsweise von SimPowerSystems:

- Die Funktion poweranalyze analysiert das elektrische Netzwerke und erzeugt die Matrizen für die Zustandsdarstellung des linearen Teilsystems.
- Unterhalb der Blöcke (als Subsysteme) werden „Standard“-Simulink-Strukturen angelegt, die das Systemverhalten darstellen. Diese Teilsystem sind mittels Goto/From-Blöcken (unsichtbar) miteinander und mit den relevanten (Simulink) Ein- und Ausgängen verbunden, während die „elektrischen“ Verbindungen nur noch zur Darstellung dienen.
- Unterhalb der PowerGUI liegt das lineare Teilsystem in Zustandsdarstellung und unter den aktiven Komponenten (Dioden, Schalter, usw.) die zugehörigen nichtlinearen Teilmodelle.



6. Simulink - 6.8 Zusatzbibliotheken

Stateflow:

